

面向边缘智能的 AIOS 设计与优化

在 Starry 上承载 RK3588 网球拾取机器人
视觉流水线的实现与系统级优化

赛题编号	proj4
赛题类型	工程型（难度 A）
目标平台	OrangePi 5 Plus / RK3588
参赛队伍	Posad
参赛学校	清华大学
队 员	洪世金、王乙凡、徐子航

摘要

本工作面向边缘智能场景下的具身智能应用，将一台基于 RK3588 的开源网球拾取机器人的操作系统，由基于 C 语言的 Linux 迁移至以 Rust 语言实现的 Starry。该机器人由相机采集图像，经 YOLOv8（一种单阶段实时目标检测网络）模型在片上 NPU（神经网络处理单元，专用于神经网络推理的硬件加速器）上完成目标检测，进而驱动机械臂与底盘完成拾球。

迁移工作分两个层次。前者补齐 Starry 运行该负载所缺的系统能力，包括块设备完成路径的容错、机械臂控制器所用的 USB 串口驱动、底盘电机所用的 PWM 控制与 reboot 系统调用，以及 Starry 此前没有的 RGA（二维图像缩放与色彩转换）和 JPU（JPEG 硬件解码）两个加速器驱动，使同一份可执行文件在 Starry 上得到与 Linux 一致的检测结果。后者建立可测量的剖析口径，在同一份程序上以 Linux 为基线逐阶段定位开销，并以一组优化将端到端推理时延逐步压低，最终在与相机输出一致的输入尺寸下达到 11.75 ms，与 Linux 的 11.4 ms 处于同一水平。

工作过程中形成的若干通用能力已在内核中实现，其中基于硬件性能监控单元的原生 perf 工具与 RGA 驱动已作为合并请求进入主线仓库；面向边缘设备图形显示的一组内核能力，涵盖显示、输入、套接字与事件循环，亦已合并并使参考的 Weston 合成器得以在 Starry 上运行。

目录

一、 引言	4
二、 基础版本与增量贡献	5
三、 系统总览	6
四、 视觉感知流水线	7
(一) USB 相机采集	8
(二) JPU 硬件 JPEG 解码	10
(三) RGA 二维加速	12
(四) NPU 推理与 RKNN 运行时	14
(五) 零拷贝与 dma-heap	16
五、 机器人本体与控制	18
(一) 差速底盘与电机控制	18
(二) 机械臂控制	19
(三) 拾球控制状态机	21
六、 支撑机器人运行的系统能力	22
(一) 启动与根文件系统	22
(二) 大小核调度与负载均衡	24
(三) 基于硬件 PMU 的 perf	26
(四) USB 串口与 reboot 系统调用	27
七、 性能优化	29
(一) 剖析方法与测量工具	29
(二) 性能概览	30
(三) 用户态优化	31
(四) 内核侧的优化支撑	34
八、 面向边缘设备的图形与显示支持	35
九、 面向 AIOS 的 AI 辅助开发方法	37
十、 队员分工	39

十一、 开发过程与时间线	40
十二、 可复现性与后续工作	40

1 引言

近年来，边缘智能在 AIoT、无人系统与机器人等领域快速发展，并逐步从实验环境走向复杂的真实场景。以基于 LeRobot 框架的自主网球拾取机器人为例，此类应用将高频视觉感知、实时决策与运动控制集成于一台算力受限的设备之上，对系统的时延、抖动与资源利用率均提出了较高要求。

目前此类应用大多构建于 Linux 之上。Linux 凭借完善的驱动与软件生态显著降低了开发门槛，但其面向通用场景的体系结构，在边缘设备上引入了若干并非必要的开销，例如冗余的系统服务、较长的执行路径、较为复杂的调度，以及相对可观的内存占用。在算力受限的条件下，这些开销会反映到机器人的端到端时延与抖动之上。一个针对该场景深度定制的操作系统，因而有机会更充分地释放硬件性能。

Starry 是一个以 Rust 语言实现、构建于 ArceOS 组件之上的操作系统。本工作的目标是上述网球拾取机器人的内核由 Linux 迁移至 Starry，在 RK3588 开发板上实现或补全运行所需的系统调用、文件接口与关键外设驱动，打通基于 NPU 的推理链路，并以 Linux 为基线，在端到端时延、启动速度、推理帧率、资源占用与能耗比等方面开展系统级优化。

本文把这台机器人当作一个完整的系统来呈现。第三节给出硬件平台与端到端数据通路；第四节自相机到 NPU 逐一展开视觉感知流水线的各个部件；第五节是底盘、机械臂与控制状态机这一机器人本体；第六节汇集支撑其运行的系统能力；第七节以 Linux 为基线开展性能优化。在视觉负载之外，第八节是一项面向边缘设备图形显示的系统级工作。我们也把 AI 辅助开发本身作为一项方法上的产出加以总结，见第九节。

图 1 概括了本工作的优化历程，从最初配置出发逐步逼近 Linux，各步的具体数据在第七节展开。

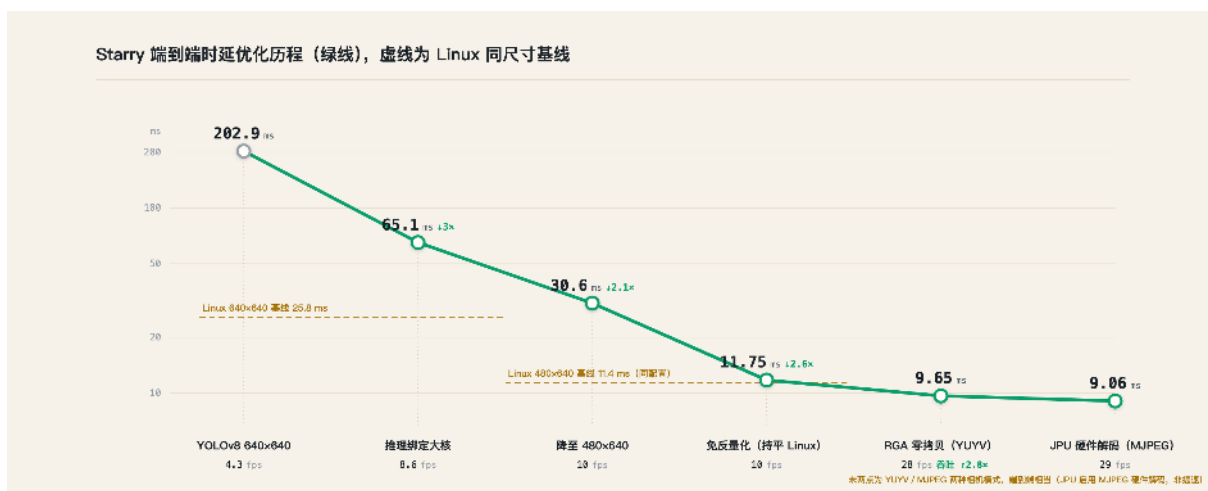


图 1 优化历程示意。前段为 640×640 输入（对照 Linux 25.8 ms），自约 30.6 ms 起改用与相机输出一致的 480×640 输入（对照 Linux 11.4 ms），故曲线后段的下降兼含输入尺寸的变化

为使两侧的比较具有意义，全篇采用同一份遵循 Linux ABI 的可执行文件。该文件在 Linux 与 Starry 上加载相同的推理运行时库 `librknnrt.so` 与相同的模型权重，使观察到的差异尽可

能归结于操作系统本身。

2 基础版本与增量贡献

本工作在若干开源项目的基础上展开,相关依赖在首次提交中即予标注。操作系统内核基于 rcore-os/tgoskits 的 dev 分支 (commit 73409e079, 2026-06-22); 机器人应用移植自 pengzechen/aka-rk3588, 我们将其改造为一个 tgoskits 应用; 模型转换使用 airockchip/rknn-toolkit2。本队在此基础上的增量贡献见表 1, 其中多项已作为合并请求提交至上游仓库 rcore-os/tgoskits。

表 1 本队的增量贡献, PR 均提交至上游仓库 rcore-os/tgoskits

贡献	说明	上游提交
硬件 PMU perf	按任务到大小核的原生 perf	PR #1395, 已合并
USB 串口驱动	机械臂控制器的串口通信	PR #1378, 已合并
reboot 系统调用	加速 Linux 与 Starry 的切换	PR #1358, 已合并
rknpu 整合与缓冲修复	DRM ioctl 整合、GEM 映射修复	PR #1351、#1364, 已合并
图形与显示内核能力	DRM/KMS、输入、套接字与事件循环, 支撑 Weston	PR #503 至 #516、#1160 等, 已合并
RGA 驱动	/dev/rga, 对齐 librga 接口	PR #1388, 开放中
JPU 驱动	/dev/mpp_service, 对齐 MPP 接口	PR #1456, 开放中
块设备完成路径容错	使开发板得以稳定挂载启动	待提交
PWM 驱动	底盘电机的 sysfs PWM 控制	待提交
profile-rknpu 计时特性	NPU ioctl 各阶段计数	待提交
大小核负载均衡器 (原型)	架构感知调度的基础	待提交
剖析工具链与基准应用	同一 ELF 的可测量剖析	随应用提交

3 系统总览

这台机器人由相机持续采集图像，经预处理后送入片上 NPU 运行 YOLOv8 检测网球，控制状态机检测结果驱动差速底盘趋近目标、指挥机械臂抓取，再寻找回收桶并放球，如此往复构成一个从感知到执行的闭环。整机的硬件部件、片上的计算单元与其上的软件分层之间的关系见图 2。

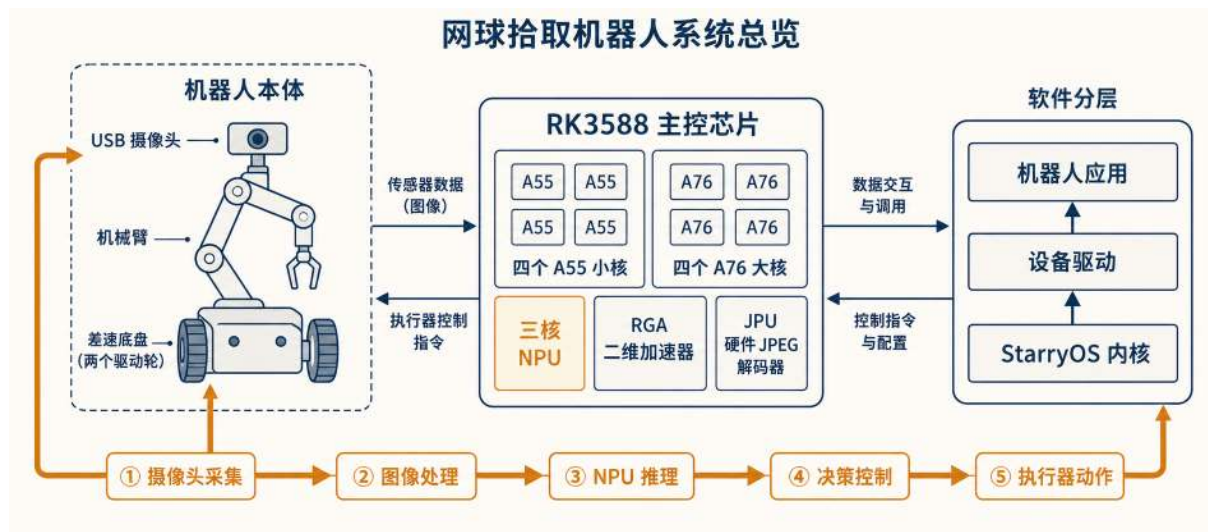


图 2 网球拾取机器人系统总览，左侧为本体部件，中部为 RK3588 主控芯片的计算单元，右侧为 Starry 上的软件分层，一条数据流自相机采集贯穿推理与决策直至执行器动作

测试硬件为 OrangePi 5 Plus 开发板，主控为瑞芯微 RK3588。其处理器采用大小核异构结构（ARM big.LITTLE），含四个 Cortex-A55 小核，主频 1.8 GHz，以及四个 Cortex-A76 大核，主频 2.256 GHz。片上集成三核 NPU，单核主频 1.0 GHz；另有硬件 RGA（Raster Graphic Accelerator，瑞芯微的二维图形加速引擎，可完成图像缩放、色彩空间转换与图层拼接等操作）与硬件 JPU（此处指 RK3588 的硬件 JPEG 解码单元 VDP720，设备地址 `jpegd@fdb90000`）。内存为 8 GB LPDDR4X。这些计算单元彼此独立，能否让感知、解码与推理各自落到合适的硬件上而不彼此争抢，是本职工作性能能否逼近 Linux 的关键。

机器人程序的视觉流水线由若干阶段串联构成。相机的输出有两种格式，YUYV 是未压缩的像素流，只需做色彩空间转换即可得到 RGB；MJPEG 是逐帧 JPEG 压缩的视频流，需先经 JPEG 解码再转换。得到 RGB 后，图像经等比缩放与填充得到模型输入，随后写入 NPU 完成推理，取回输出做后处理得到检测框，最终交由控制环驱动机械臂与底盘完成拾取。整条流程如图 3 所示，从相机采集到机械臂执行构成一个闭环。表 2 给出各阶段的命名与职责，是后文逐阶段数据与剖析所用的统一口径。两侧加载的是同一份模型权重，因此在功能层面具备可比性。所谓 RKNN，是瑞芯微 NPU 所用的运行时与模型格式，常见框架训练出的模型须先经 rknn-toolkit2 工具转换为该格式方可在 NPU 上加载。

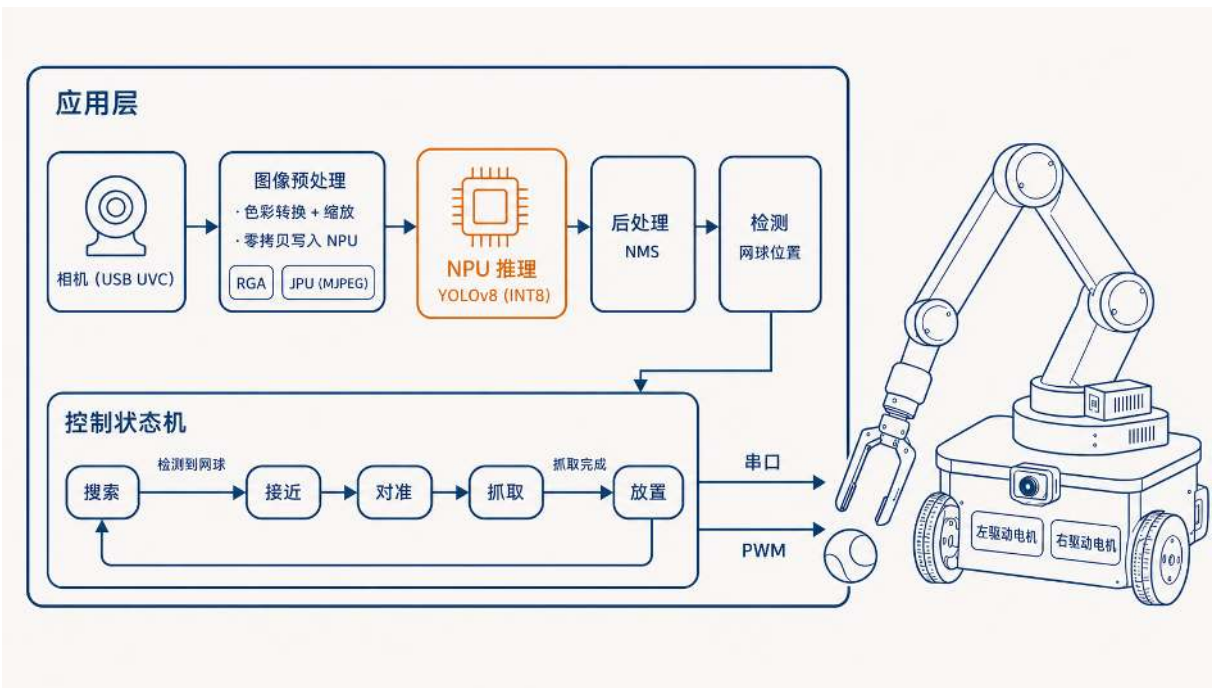


图 3 用户态应用的完整工作流程，相机采集经图像预处理与 NPU 推理得到网球位置，控制状态机据此驱动机械臂与底盘完成拾取

表 2 视觉流水线的阶段划分

阶段	职责
decode	将相机帧由 YUYV 或 MJPEG 解码为 RGB
letterbox	等比缩放并填充至模型输入尺寸
inputs_set	将输入写入 NPU，含必要的缓存维护
run	NPU 完成一次前向推理
outputs_get	取回 NPU 输出，并把其通道分块的特殊排布整理成常规张量（张量重排）
postprocess	把网络输出解码为边框坐标（DFL），再用非极大值抑制（NMS，合并高度重叠的候选框）得到检测框

4 视觉感知流水线

视觉感知是这台机器人最重的负载，也是它与边缘智能这一主题最直接相关的部分。一帧图像自相机进入，经解码、缩放、推理与后处理，最终化为一个网球的位置。这一路上的每一步都可能落到 CPU 上拖慢整体，也都可对应到 RK3588 的一个专用单元。下文自相机起，逐一展开这条流水线上的部件，先讲清各自的架构，再说明本队在 Starry 上把它跑通、跑快所做的工作，各部件之间如何共享内存以避免拷贝则集中在第（五）节。

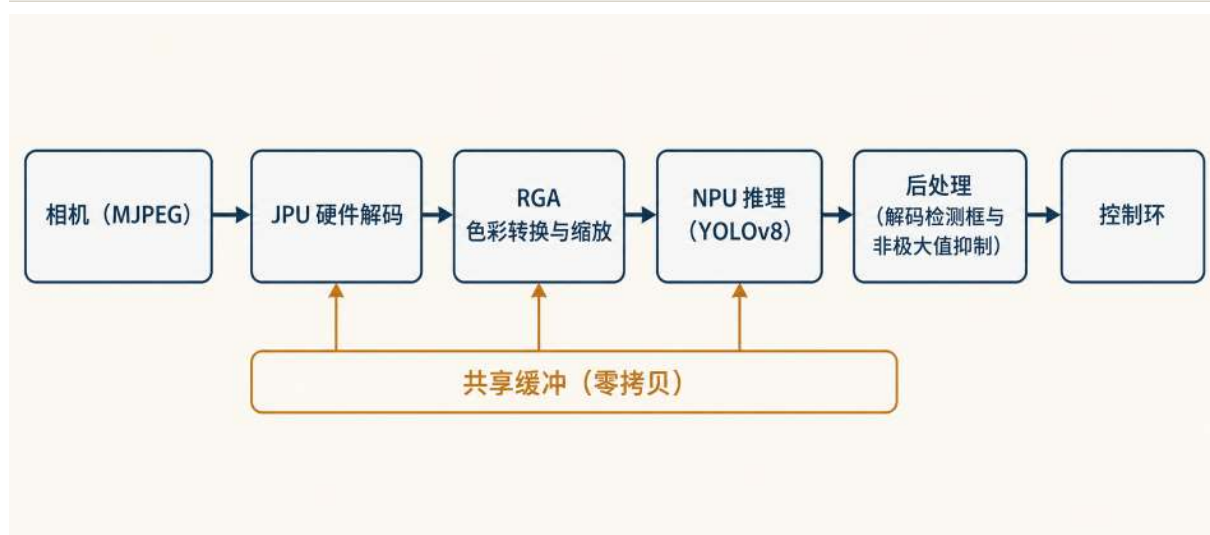


图 4 相机到控制的视觉流水线，各处理阶段自左向右串联，JPU、RGA 与 NPU 三者读写同一段共享缓冲，阶段之间不再发生内存拷贝

4.1 USB 相机采集

整条感知流水线的第一帧来自一只接在 USB3 端口上的相机。它属于 USB 视频类（UVC，一套把摄像头的取景、格式协商与帧传输标准化的设备规范），因此无需专用驱动，任何符合该类的相机都以同一套描述符与控制请求被识别和驱动。这只相机能交付两种格式，一种是 YUYV，即未经压缩、按 4:2:2 排布的原始像素，取来只需做一次色彩空间转换；另一种是 MJPEG，每帧是一张独立的 JPEG，取来还要先解码。两种格式后续都进入 RGA 与 NPU，区别只在前端是否多一步解码。

相机的像素流不像存储或网络那样按需搬运，它是连续不断、对时延敏感而对个别丢失不敏感的。USB 为这类负载准备了等时传输（一种周期性、预先预约总线带宽、且不做重传的传输方式）。主机在配置端点时就为它在每个服务周期里划定一份固定带宽，到点必送，送不下的那一份直接丢弃而非排队重发，于是时延有上界，代价是允许偶发的帧内字节缺失。这正合视频流的需要，一帧迟到的画面对机器人毫无意义，宁可丢掉去取下一帧新的。承载这条流的是 RK3588 上的 DWC3 控制器，它在主机模式下对软件呈现为一个标准的 xHCI（可扩展主机控制器接口）主机，相机的枚举、端点配置与等时传输都经由 Starry 既有的 xHCI 驱动完成，本部件因此复用这套主机栈，把工作集中在让等时端点在这块硬件上正确地周期取数。

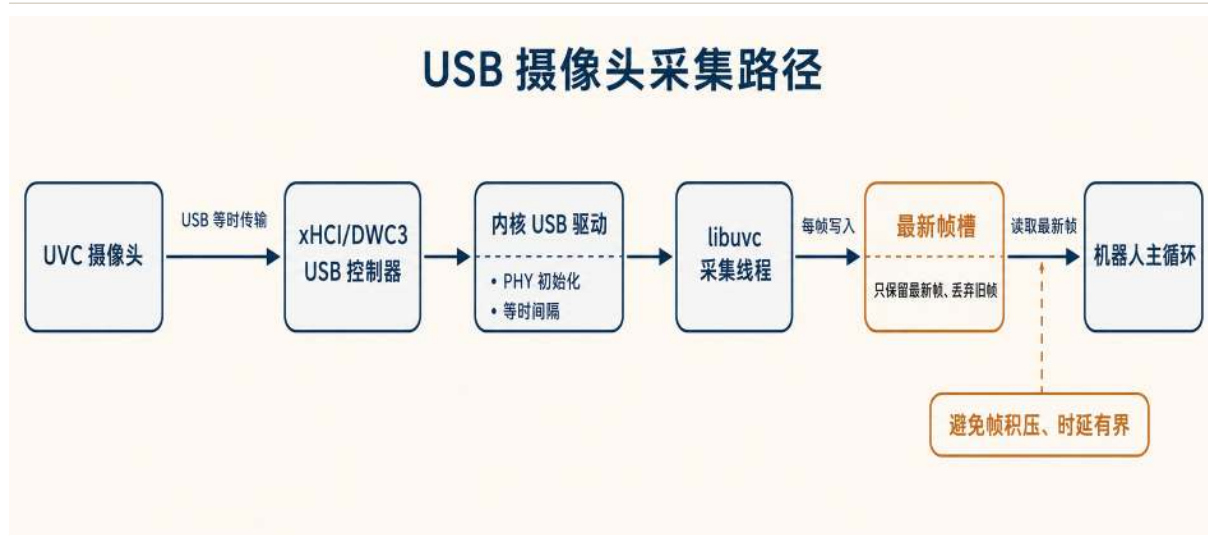


图 5 相机经 USB3 端口与 DWC3/xHCI 主机栈，以等时传输把帧送入“最新帧”槽，感知循环从槽里取走最新的一帧。

正确配置等时端点 xHCI 用一份端点上下文描述每个端点应当如何被周期服务，其中的 `Interval` 字段决定控制器多久为该端点取一次数据，它并不直接等于描述符里的 `bInterval`，而要按照端点类型与连接速度换算。对高速及更高速度的等时端点，正确的换算是 `Interval` 取 `bInterval` 减一，含义是每 2^{Interval} 个 $125\mu\text{s}$ 微帧服务一次。若图省事改用一个固定下界把它夹到 1，那么 `bInterval` 为 1 的高带宽端点会被编码成每两个微帧才服务一次，可用带宽当场减半，相机的内部缓冲随即溢出，等时帧被成片截断。换算因此严格按规范分速度分类类型计算，`usb-host` 的 `calculate_xhci_interval` 把这一点固定下来。同一段端点配置里，等时端点的错误计数被显式置零，因为等时传输本就不重传，给它配置重试次数没有意义；中断与等时端点还按每微帧包数算出突发大小与一次服务区间的最大载荷（max ESIT payload），一并写入上下文，控制器才会按相机申报的带宽真正预约总线。

用传输环逐包提交一帧 一帧的数据不是一次提交，而是被切成多个等时数据包，每个包对应传输环（一段主机与设备共享、由 TRB 描述待办传输的环形缓冲）上的一个等时 TRB。`usb-host` 为每个包按相机的最大包大小与突发参数算出突发计数与末段包数填入 TRB，逐包串成一个传输描述符提交给控制器，敲一次门铃即开始周期取数。设备每完成一个包就回送一个传输事件，驱动据其完成码与残余长度算出该包实到字节，集齐一帧的所有包后把整帧交还上层。这里有两处与等时语义直接相关的处理。其一，当周期环被取空，控制器会上报环下溢或环上溢事件，它们并非某个传输的完成，而是“这一拍没数据可送”的提示，驱动按 Linux 的做法把这类事件当作环空跳过，不让它们被误当成帧完成。其二，对实到字节短于申报长度的等时帧，驱动逐包归类为足额、偏短或为零，并与缺失完成事件的包数分开统计，从而把“事件投递丢失”与“控制器侧限速或 FIFO 截断”两类成因区分开，这是在板上诊断一条被截断的等时输入流时唯一能凭代码坐实的判据。

用户态只保留最新的一帧 即便等时端点已正确取数，若取来的帧在用户态排队等待处理，时延仍会随队列累积而失控。采集侧因此不排队。基于 libuvc 的采集线程独立运行，每收到一帧就进入一个互斥锁保护的“最新帧”槽，把帧序号、尺寸、格式与像素数据整体写入该槽并覆盖其中原有的一帧。覆盖之前若上一帧尚未被取走，只把一个丢弃计数加一，绝不把旧帧留下排队。感知循环从另一侧读取，先在锁内只看一眼槽里的帧序号，序号未变就直接返回、不做任何拷贝；仅当序号变化才把这一帧拷出并打上采集时刻的时间戳。如此一来，无论感知与控制这一侧快慢，它每次拿到的都是当下最新的一帧，旧帧被持续丢弃，端到端时延不被采集积压拖长。这一“丢旧取新”的纪律由 `tennis-app` 的采集封装与 `Camera::poll` 共同维持，丢弃帧数也作为基准指标的一项被记录。

4.2 JPU 硬件 JPEG 解码

机器人用的 USB 相机有两种交付像素的方式。一种是未压缩的 YUYV，整帧逐像素送出，占满带宽；另一种是 MJPEG，即把视频的每一帧各自压成一张独立的 JPEG 图片再传出，体积小得多，因而是这类相机在较高分辨率与帧率下的原生工作模式。代价是接收端拿到的每一帧都还是一张压缩的 JPEG，必须先解码还原成像素才能进入后续处理。若由 CPU 来做这件事，一帧的 JPEG 解码在本平台上约需二十五毫秒，几乎与一次推理相当，整条流水线会被这一步拖垮。RK3588 为此集成了一个专用的硬件 JPEG 解码单元，即 VDP720（设备树节点 `jpegd@fdb90000`，本文称其为 JPU），它在硬件中完成熵解码与反变换，直接把一张 JPEG 还原为 NV12 排布的 YUV 帧并写入物理内存，使这一步从计算路径上让出。

在 Linux 上，应用并不直接操作这块硬件，而是经由瑞芯微的多媒体处理平台 MPP（Media Process Platform）库。MPP 把“解码一帧”抽象成一次会话，应用通过 `librockchip_mpp` 提交压缩数据、取回解码帧，库再经字符设备 `/dev/mpp_service` 把寄存器配置投向内核。本队在 Starry 上重建了 `/dev/mpp_service` 这一节点，实现了 `librockchip_mpp` 的 JPEG 解码路径所用的那一小部分 `ioctl` 约定，使未经改动的 MPP 程序，包括其标准解码测试 `mpi_dec_test` 与 `gststreamer`、`ffmpeg` 的瑞芯微后端，都能照常驱动这块解码器。平台无关的驱动核心做成一个 `#![no_std]` 的 crate，负责把寄存器文件、量化表与霍夫曼表的字节排布按厂商 `hal_jpegd_rkv` 的方式拼出来；设备节点这一层只做用户态数据的拷入拷出、把 `dma-buf` 文件描述符翻译成物理地址，以及真正驱动一次硬件运行。

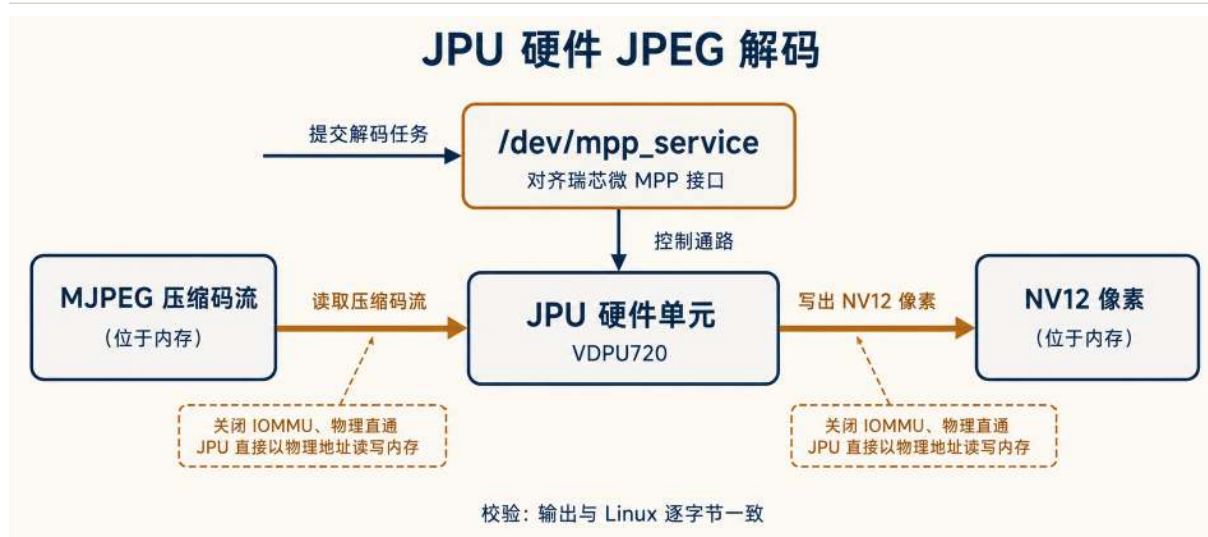


图 6 相机 MJPEG 帧经 librockchip_mpp 与 `/dev/mpp_service` 进入 JPU 硬件解码，输出 NV12 帧落在与 RGA、NPU 共享的同一段 dma-buf 内存上

对齐 MPP 的会话协议 MPP 与内核之间并非一次函数调用，而是一串按字节对齐、经 `ioctl` 传入的请求记录。每条记录是一个 24 字节的结构，带有命令号、标志位与一个指向各自负载的用户态指针，多条记录以 `MULTI_MSG` 标志串成一批，到 `LAST_MSG` 为止。会话先以 `PROBE_HW_SUPPORT` 询问本节点支持哪类编解码，节点据实回报只支持 JPEG 解码；再以 `QUERY_HW_ID` 读硬件版本，并以 `INIT_CLIENT_TYPE` 把会话绑定到 JPEG 解码这一客户端类型，绑定到其它类型会被拒绝。随后 `SET_REG_WRITE` 送来要写入硬件的整组寄存器，`SET_REG_READ` 声明完成后要读回的寄存器窗口，最后 `POLL_HW_FINISH` 触发一次提交并阻塞到这一帧解码完成。节点刻意不去提供查询命令支持表的接口，从而让 MPP 走它对旧内核的兼容回退路径，恰好对上本节点实现的这一组命令，使既有的 MPP 二进制无需任何改动。

把文件描述符翻译成解码器看得见的地址 MPP 在寄存器组里并不直接写物理地址，而是把缓冲的 dma-buf 文件描述符放进几个约定的地址寄存器槽，另以 `SET_REG_ADDR_OFFSET` 附带每个槽的字节偏移。提交前节点逐一扫描这些地址槽，按厂商 `trans_tbl_jpgdec` 给定的位置，把描述符经共享的 `/dev/dma_heap` 解析为其连续缓冲的物理基址，再加上声明的偏移写回寄存器。量化表、霍夫曼最小码表与霍夫曼值表共处一段表缓冲，分别落在偏移 384 与 704 字节处，输入码流与输出帧各占一个槽，于是 MPP 用文件描述符表达的一组缓冲被还原成解码器能直接取用的物理地址。

让引擎在不开启地址翻译的情况下取到数据 这块解码器的前面本来挂着一个属于它自己的小型地址翻译部件，即 IOMMU，作用是把设备发出的虚拟地址翻译成内存中的物理地址。开启它需要为设备维护一套页表，而本流水线里所有缓冲都来自 `/dev/dma_heap`，在四吉字节以下连续分配，物理地址本身就是连续的，无需翻译。因此设备初始化时把这个 IOMMU 直接置为直通：它的寄存器在同一寄存器页偏移 0x480 处，先写入强制复位命令清掉前一个系统可能遗留的页表，再把页表基址寄存器 `RK_MMU_DTE_ADDR` 清零、写入停用分页命令 `RK_MMU_CMD_DISABLE_PAGING`，

最后把它的中断全部屏蔽。此后引擎按物理地址直接读写内存，`dma_addr` 与物理地址、设备可见地址三者相等。

先配置寄存器，再启动引擎 解码器的启动位与状态位同处控制寄存器 `SWREG1`。运行时把整组寄存器写入硬件时，刻意跳过只读的 `ID` 寄存器，并把带启动位的 `SWREG1` 留到最后一个写，从而保证几何、格式、表长与五个地址槽都就位之后引擎才被拉起，绝不会在输入尚未配齐时就开始取数。完成与否由轮询 `SWREG1` 判断，其中分别有解码就绪位与总线错误、码流错误、超时、码流为空几个错误位；读回整组寄存器供诊断之后，无论成功、出错还是超时，都以写一清零的方式清掉这些状态位，使陈旧的完成或错误标志不会被下一帧误读。完成信号这里以轮询观察，无需真正布线中断。

三个加速器共享同一段内存 JPU 在本流水线里不孤立工作，它的输出正是下一级 RGA 色彩转换与缩放的输入，而 RGA 的结果又直接写入 NPU 的输入缓冲。这要求三者读写同一段物理内存而非各自拷贝。承载解码输出的 NV12 帧由 `/dev/dma_heap` 在四吉字节以下连续分配，以 `dma-buf` 文件描述符在 JPU、RGA、NPU 之间传递，JPU 解出的帧落在这段缓冲上即可由 RGA 按描述符直接读取，整条 MJPEG → JPU → RGA → NPU 通路因此全程零拷贝。解码器是 32 位寻址，节点在解析文件描述符时拒绝四吉字节以上的缓冲而非默默截断地址，恰与 `dma_heap` 的分配区间一致。

驱动核心的寄存器组、量化表与霍夫曼表拼装在主机上有完整的单元测试，包括对照厂商布局逐字节比对表缓冲、断言启动位写在最后、以及量化表的反之字形重排。在开发板上，标准的 MPP 解码测试 `mpi_dec_test` 解出的结果与 Linux 逐字节一致，校验和为 2712426383，单帧解码的中位延迟约一毫秒。需要说明的是，JPU 的意义并不在于比 YUYV 色彩转换路径更快，二者并不可比；它的意义在于为相机的原生 MJPEG 模式提供一条硬件解码通路，使这一步不必回落到约二十五毫秒的 CPU 软解。接入之后，MJPEG → JPU → RGA → NPU 全程零拷贝，端到端中位延迟为 9.06 ms，约合每秒二十九帧，此时的瓶颈已是相机供帧速率，与 YUYV 路径的 9.65 ms 处于同一水平。**该工作以合并请求 #1456 提交至上游。**

4.3 RGA 二维加速

机器人采集到的每一帧并不能直接送入检测网络。相机交付的是 YUV 排布的像素，而 YOLOv8 的输入要求是连续的 RGB 平面，尺寸还需缩放到模型的输入分辨率。若交由 CPU 逐像素完成色彩空间转换与缩放，一帧的开销与一次推理相当，感知与计算彼此争抢同一个核，流水线随即被这一步拖住。RK3588 为此集成了 RGA 二维栅格图形加速器，它在硬件中完成色彩空间转换、缩放与图层拼接，并能直接读写物理内存，使这一步从计算路径上让出。

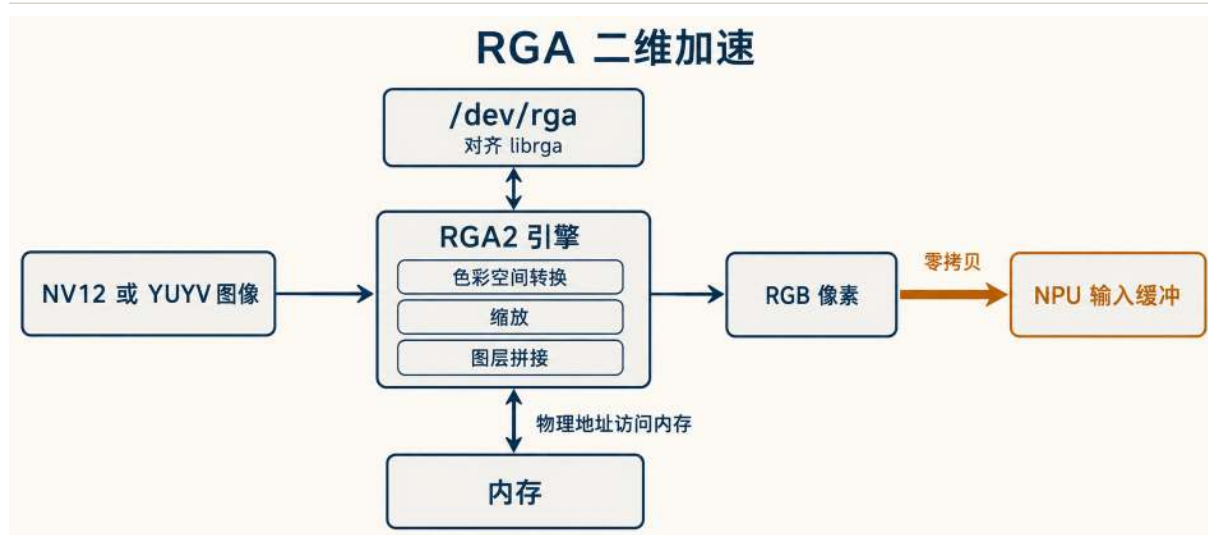


图 7 RGA 二维加速的软件栈与数据通路，librga 经 `/dev/rga` 提交 `rga_req`，内核翻译为命令块交由 RGA2 引擎执行，结果直接写入下游缓冲

RGA 的软件入口在 Linux 上是 `/dev/rga` 与用户态的 librga。应用以一份 `rga_req` 描述源与目标的地址、格式、尺寸与裁剪窗口，经一次 `RGA_BLIT_SYNC` 同步阻塞调用提交，内核把它翻译为一段命令块，交由 RGA2 引擎执行。本队在 Starry 上重建了这条通路，一个平台无关的 `#![no_std]` 驱动核心负责命令块编码与寄存器时序，`/dev/rga` 设备节点对齐 librga 的 `ioctl` 约定，使既有的 librga 二进制无需改动即可驱动这块硬件。

对齐用户态的二进制接口 librga 与内核之间并非函数调用，而是一份按字节对齐的结构体经 `ioctl` 传递，任何一处字段错位都会让引擎读到错误的地址或尺寸。`rga_req` 在 LP64 下恰为 504 字节，其中嵌入三份各 56 字节的图像描述符，缓冲导入用的 `rga_external_buffer` 为 288 字节，这些尺寸以编译期断言固定下来，任何字段增删都会在编译时暴露。早期的镜像漏掉了 `rotate_mode`、`rd_mode` 等几个较新的字段，使其后所有字段整体偏移，引擎据此取到错位的旋转矩阵与地址。librga 在打开设备时还会查询驱动版本与硬件版本以选择匹配的结构体布局，因此设备节点须如实报告 MultiRGA v1.3.1 与 RK3588 上 RGA2 核的版本号，librga 才会按这一布局组包并把请求投向 RGA2 而非另一代核心。

在三个核中找到正确的那一个 RK3588 的设备树把 RGA 暴露为三个独立设备，两个 RGA3 核与一个 RGA2 核。本流水线所需的色彩转换与缩放走 RGA2 路径，若按设备枚举顺序取首个核，落到的是一个没有 RGA2 引擎的 RGA3 核，提交随即以无此设备失败。设备节点因此按版本而非按枚举序检索，锁定真正承载 RGA2 的那个核再提交。

让引擎真正跑起来并报告完成 RGA2 的执行时序对寄存器写入的顺序敏感。引擎须先被置入主模式，命令起始信号才会触发一次命令 DMA 取指，停留在从模式时起始信号什么也取不到。完成信号同样有讲究，表示全部命令完成的状态位只有在对应的中断使能于起始之前被置位时才会锁存。最初省略了这一步，引擎明明已经跑完，完成位却始终为零，轮询一路超时，看上去

像是硬件挂死。据厂商时序在主模式与起始之间补上这次对中断使能位的读改写后，完成位正常锁存，既有的忙等轮询即可观察到它，无需真正布线中断。命令缓冲的基址按原始字节地址写入，不做页表基址那样的右移，这一区别在 MMU 关闭、直接使用物理地址的本路径上是必须的。

一个把缩放挂死的定点系数 缩放系数以 16.16 定点表示。缩小时正确的系数是目标维度比源维度，其值不超过一，写入低半字；最初误用了它的倒数，缩小时系数大于一，使源指针每步跨过有效区域，扫描器随即停在忙状态约五十毫秒不再前进。把缩小与放大两种情形分别按厂商参数计算后，缩放回到一次提交即完成。

让三个加速器读同一段内存 RGA 在本流水线里不孤立工作，它读上一级解码的输出，又把结果直接写入 NPU 的输入缓冲，这一共享缓冲的机制见第（五）节。RGA2 在本阶段以 MMU 关闭、物理直通的方式运行，`/dev/rga` 收到 `dma-buf` 文件描述符后解析出物理地址交给引擎，并在提交与轮询期间持有该分配的引用计数，使另一线程的并发释放不会在操作中途抽走内存，提交前后还须显式完成缓存与设备方向的同步，引擎与 CPU 才看到一致的数据。

驱动的五项基本操作在开发板上以逐像素 CRC 校验通过自测，该工作以合并请求 #1388 提交至上游。在 Linux 上的受控对照中，以 RGA 替代 CPU 缩放可将该阶段由 15.82 ms 降至 1.72 ms，约九倍；Starry 接入这条路径后，色彩空间转换的 p50 为 0.776 ms，缩放也接近归零。这一步把图像准备从计算路径上彻底让出，为后续把推理放置到大核腾出了余量，相关数据见第七节。

4.4 NPU 推理与 RKNN 运行时

经过缩放与色彩转换的那一帧最终要送进检测网络，承担这一步的是 RK3588 片上集成的神经网络加速器。它由三个独立的计算核组成，每个核运行在 1.0 GHz，专门以低精度整数算术高吞吐地执行卷积、矩阵乘与激活这类张量算子。直接在 CPU 上跑同一张网络会让感知与控制争抢通用核，一帧的代价高到流水线无法承受，把这部分计算交给专用加速器，CPU 才能腾出来去做决策与运动控制。

加速器自身只认一种以低精度整数权重编排好的指令流，普通深度学习框架训练出的模型并不能直接喂给它。瑞芯微为此提供了一套运行时与模型格式，运行时以用户态共享库 `librknnrt.so` 的形式存在，模型则由 `rknn-toolkit2` 这一离线工具把常见框架的网络转换为 RKNN 格式，转换过程顺带把浮点权重量化为 INT8 并记下每个张量的量化标度。应用通过这套运行时的接口加载模型、提交一帧、取回结果，运行时再把这些请求翻译成对加速器的命令提交与内存操作，经设备节点下达到内核。

这条路径在本系统里落在 `/dev/card1` 这个设备节点上，它在内核中表现为一个 DRM（图形显示子系统）字符设备。运行时与内核之间的交互全部以 `ioctl` 命令完成，其命令号按 DRM 的约定编码，低于 `0x40` 的是 DRM 核心命令如查询驱动版本，落在 `0x40` 到 `0xA0` 之间的才是 RKNPU 这个驱动自己的命令，节点据此把版本查询与厂商私有的内存与提交命令分派到不同的处理路径。RKNPU 的私有命令共六类，分别负责内存对象的创建、映射、销毁、缓存同步，以

及任务的动作查询与提交，对应 `rknpu_driver_ioctl` 里的 `MemCreate`、`MemMap`、`MemDestroy`、`MemSync`、`Action` 与 `Submit`。

加速器看到的输入张量并不是一段裸内存，而是经由图形子系统的缓冲对象（GEM，graphics 子系统里统一管理显存与设备内存的句柄）来描述。运行时先以一次内存创建命令换得一个句柄，再以这个句柄向内核请求把对应的物理内存映射进自己的地址空间，此后对这块张量的读写就直接落在加速器要访问的那段内存上。这层缓冲对象正是让上一级的图像加速器与本级共享同一段内存的关键，输入缓冲以句柄在两者之间传递，加速器与图像准备阶段读写的是同一块物理页而非各自的副本，整条流水线因此免去了一次拷贝。

让每次推理都能接续前一次 这个节点与其驱动在本系统里本已存在，但要真正承载一帧接一帧的连续推理，缓冲对象在两处的行为必须被修正，否则总是上一帧能跑，下一帧就失败。第一处在映射时。运行时按句柄请求映射，节点把句柄换算回缓冲对象的物理地址并交给上层去建立页表，节点据缓存策略决定这段映射是可缓存还是非缓存，写合并因为内核尚未提供对应的页表项模式而被退化为非缓存处理，以免误把它当作可缓存而读到陈旧数据。映射建立时若不正确处理这段区域的缺页，第一次访问就会触发无法收场的异常，使紧随其后的那次推理取不到输入。第二处在销毁时。每个缓冲对象都带引用计数，运行时取回结果后会销毁这一帧的临时对象，若销毁时计数处理有误，被提前回收或重复释放的那段内存会在下一帧被再次创建时拿到错位的地址。把映射时的缺页处理与销毁时的引用计数这两处都修正之后，连续推理才真正稳定下来。这部分对 RKNPU 的 DRM 接入与 GEM 缓冲对象的修复以合并请求 [#1351](#) 与 [#1364](#) 提交至上游，均已合入。

在内核侧为各阶段计时 为了弄清一帧的时间究竟花在哪个 `ioctl` 阶段，节点里加进了一个可选的计时特性 `profile-rknpu`。它在提交、内存创建、内存同步等命令的处理两端各取一次 `monotonic_time_nanos` 单调时钟，算出该阶段的微秒耗时，并用一组 `AtomicUsize` 计数器以 `Relaxed` 次序无锁地为每条记录编号，超过给定条数后便不再打印。整个计时通路不持有任何锁，因此既不改变被测路径的时序，也不会多核并发提交时相互阻塞，关掉这个特性时这些代码完全不参与编译，正常运行不付出任何代价。

在整数域筛除背景 最显著的一处优化落在取回结果这一步，做在应用一侧。加速器算完后吐出的是 `INT8` 张量，要还原成有物理意义的浮点置信度，需要按各张量的量化标度做一次反量化。一帧的检测头在三个尺度上铺开约四十万个网格值，其中绝大多数是背景，没有任何目标。若让运行时在取回时把这四十万个值统统反量化为浮点，绝大部分算力都花在了注定被丢弃的背景上。本系统把取回时的浮点化关掉，令运行时直接交回原始的 `INT8` 张量，再把置信度阈值反向量化到整数域，在整数比较中先把低于阈值的网格全部筛除，只对幸存下来的那几十个网格做反量化。反量化是单调映射，在整数域比较置信度的大小与在浮点域比较完全等价，因此输出与逐点反量化的结果逐位一致。这样一来取回结果一步从约 `12 ms` 降到约 `1 ms`，而在全部 285 张测试图上，整数域路径得到的检测框与浮点路径完全相同。

这张网络以卷积类算子为主，约七成的加速器时间都花在卷积上。它的规模不大，单帧的负

载基本压在一个计算核上，再开第二、第三个核并不会带来收益，那两个核多数时候是空闲的，因此实际推理始终只用到一个核，加速器全程保持在 1.0 GHz。也正因为时钟固定在最高频不做动态降频，推理这一步（run）在本系统上与 Linux 在同一时钟下处于同一水平，约 4.7 ms，并不更快，真正被压下来的是它两侧的取回与内存操作，而非加速器的计算本身。

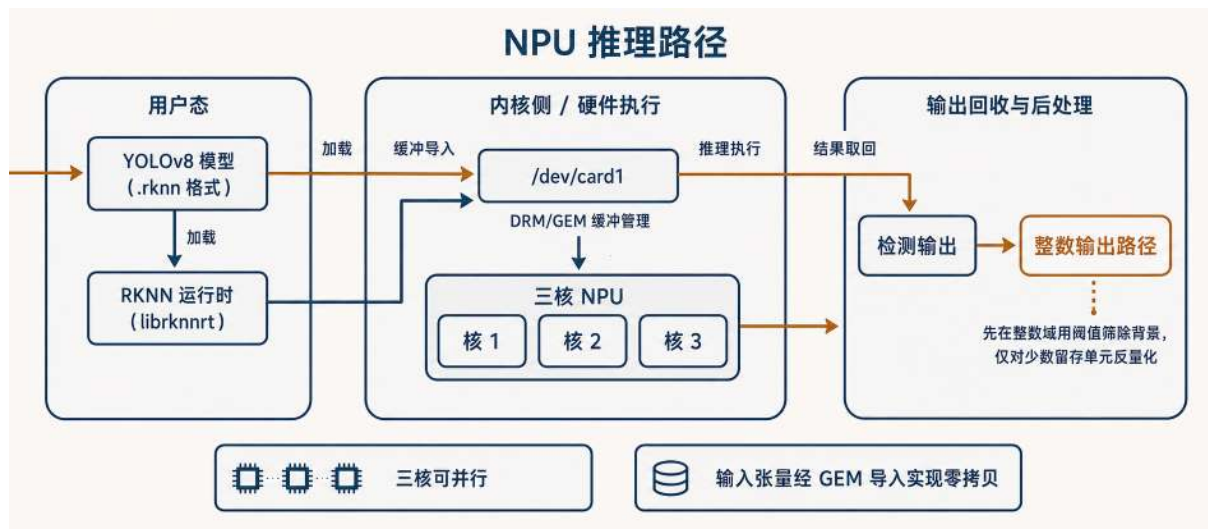


图 8 运行时经 `/dev/card1` 提交一帧的路径，输入张量以 GEM 缓冲对象在图像准备阶段与加速器之间零拷贝共享，取回结果在整数域筛除背景后只对幸存网格反量化

4.5 零拷贝与 dma-heap

一帧网球画面在送入检测网络之前要经过三个互不相同的硬件部件，先由 JPU 把 MJPEG 解码成 NV12，再由 RGA（（三））把 NV12 转换并缩放为 RGB，最后由 NPU（（四））把 RGB 读作推理输入。这三个部件各自有独立的总线主控通道，本可以各读各的内存；倘若每一级都把上一级的结果先搬回 CPU、再拷贝给下一级，单帧仅在两次搬运上就要付出与一次推理相当的内存带宽与缓存抖动，感知流水线会被这些纯拷贝拖住，而它们不产生任何有用计算。让三个加速器读写同一段物理内存、彼此之间不经过 CPU，是整条流水线能够流动起来的前提。

Linux 为这种跨设备共享提供的机制是 dma-buf 与 dma-heap。dma-heap 是一个内核内的缓冲分配器，对应 `/dev/dma_heap` 下的设备节点；应用向它请求一段内存，拿到的不是普通指针，而是一个文件描述符，这个描述符代表的缓冲可以被传给任意设备去导入并直接访问。一段内存因此只分配一次，由这个描述符在 JPU、RGA、NPU 之间传递，每个部件凭描述符解析出同一个物理地址，谁也不必再拷贝。本队在 Starry 上实现了这套机制，使既有的 Rockchip 用户态库无需改动即可把缓冲在三个加速器之间流转。

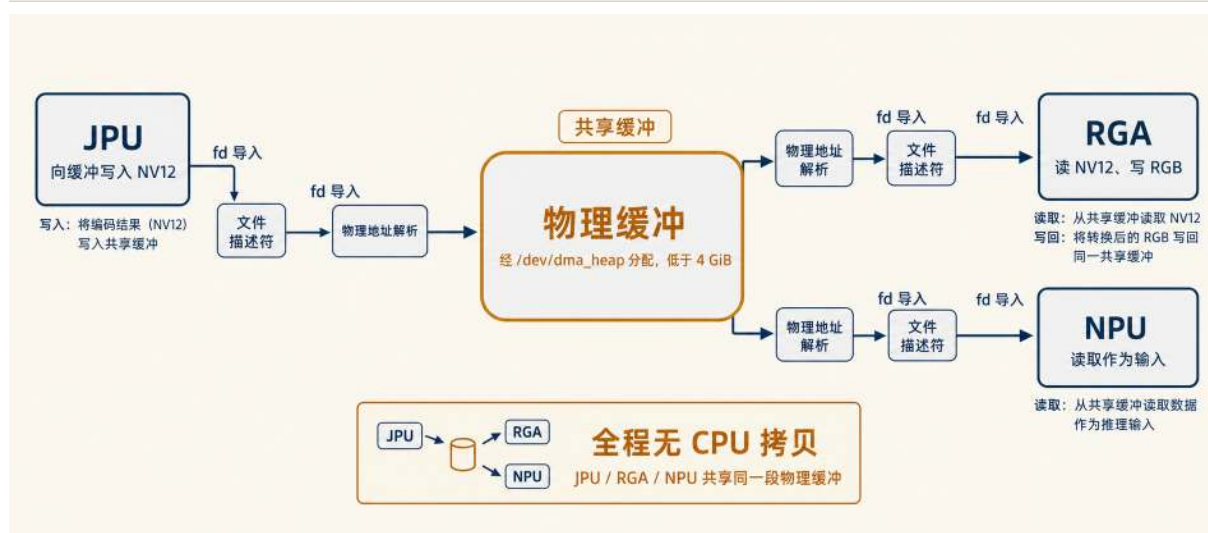


图 9 同一段经 `/dev/dma_heap` 分配的物理连续缓冲以 `dma-buf` 描述符在 JPU、RGA、NPU 之间传递，三者就地读写而不经 CPU 拷贝

分配器交出的是什么样的缓冲 设备节点 `/dev/dma_heap/system` 与 `/dev/dma_heap/cma` 共用同一个分配器，应用以一次 `DMA_HEAP_IOCTL_ALLOC` 提交一份 `dma_heap_allocation_data` 请求长度，内核返回一个可 `mmap`、可被各设备导入的 `dma-buf` 描述符。这份结构体在 LP64 下恰为 24 字节，`ioctl` 编号由内核侧按 `_IOWR('H', 0, ...)` 推算并以编译期断言固定为 `0xC0184800`，与主线 Linux 的取值逐位一致，任何字段或编号的偏移都会在编译时暴露，使用户态库据以组包的布局不会悄然漂移。分配本身落在 4 GiB 以下的物理空间，按页（4 KiB）对齐且物理连续。这两条约束并非随意而来。物理连续是因为 RGA2 与 NPU 在本阶段以设备侧地址翻译部件（IOMMU）旁路、物理直通的方式工作，它们沿一个起始物理地址顺序访问整段缓冲，中途若被分页打断便会读到错误的页；32 位地址上限则对应 RGA2 把一个裸 32 位物理基址直接写入自身寄存器的约束，因而分配器以 `u32::MAX` 作为 DMA 掩码，并把请求长度限制在 32 位以内。

同一段内存上的三个读者 JPU 把解码出的 NV12 写入这段缓冲，RGA 以 NV12 读入、RGB 写出，NPU 再把 RGB 读作输入，全程没有一次 CPU 拷贝。串起这三者的是同一个 `dma-buf` 描述符，每个部件通过导入它而拿到那个唯一的物理地址。RGA 一侧的导入入口在 `/dev/rga` 收到一个描述符或一个先前登记的句柄，经 `resolve_dmabuf_fd` 把它解析回那段分配，取出物理地址交给引擎。关键在于这次解析同时克隆了该分配的引用计数，并把这份克隆在整个提交与轮询期间持有；缓冲的生命周期由这个引用计数管理，最后一个持有者释放时页面才归还。这样即便另一线程在引擎运行途中并发地释放同一缓冲，内存也不会被中途抽走，引擎读写的始终有效的页。

让 CPU 与设备看到同一份数据 设备物理直通带来一个缓存一致性问题。这段连续缓冲在 aarch64 上是带缓存的，分配时由 CPU 清零，那些清零的缓存行处于脏状态；若不先把它写回内存，引擎读到的将是旧值，或一次稍后的缓存逐出会覆盖掉引擎刚写出的结果。因此在把缓

冲交给引擎之前，沿写入设备的方向显式同步一次，将脏的 CPU 缓存行刷回内存；待引擎完成、在读取结果之前，再沿读回 CPU 的方向同步一次，使目标缓冲对应的缓存行失效，CPU 随后读到的是引擎真正写入内存的数据而非陈旧的缓存。这两次方向相反的同步分别在引擎起始之前与轮询到完成之后进行，只针对实际参与本次操作的源与目标缓冲，CPU 与设备据此看到一致的内容。

这一层是 RGA（（三））、JPU（（二））与 NPU（（四））三节共同依赖的底座，它把原本散落在各级之间的拷贝从流水线上彻底移除，使一帧只被分配一次、被三个加速器依次就地读写。

5 机器人本体与控制

视觉给出网球的位置之后，机器人要靠本体的底盘与机械臂把球真正拾起。底盘负责趋近，机械臂负责抓取与释放，二者由一个控制状态机按当下的感知结果统一调度。下文展开这三部分，它们把第四节得到的检测结果转化为电机与舵机的动作，闭合从感知到执行的回路。

5.1 差速底盘与电机控制

机器人靠两个直流电机驱动，一侧一个，并没有独立的转向舵机。这种结构称为差速驱动，前进靠两侧轮速相同，转向则完全来自两侧轮速之差。两轮同速向前则直行，一侧快一侧慢则向轮速较低的一侧偏转，两侧反向旋转则原地转身。感知给出的目标是球或桶在画面里的位置，控制要做的就是把这个横向偏差换算成一对左右轮速，让机器人一边逼近目标一边把它拢到画面中央，最终停在可以伸臂抓取的位置。

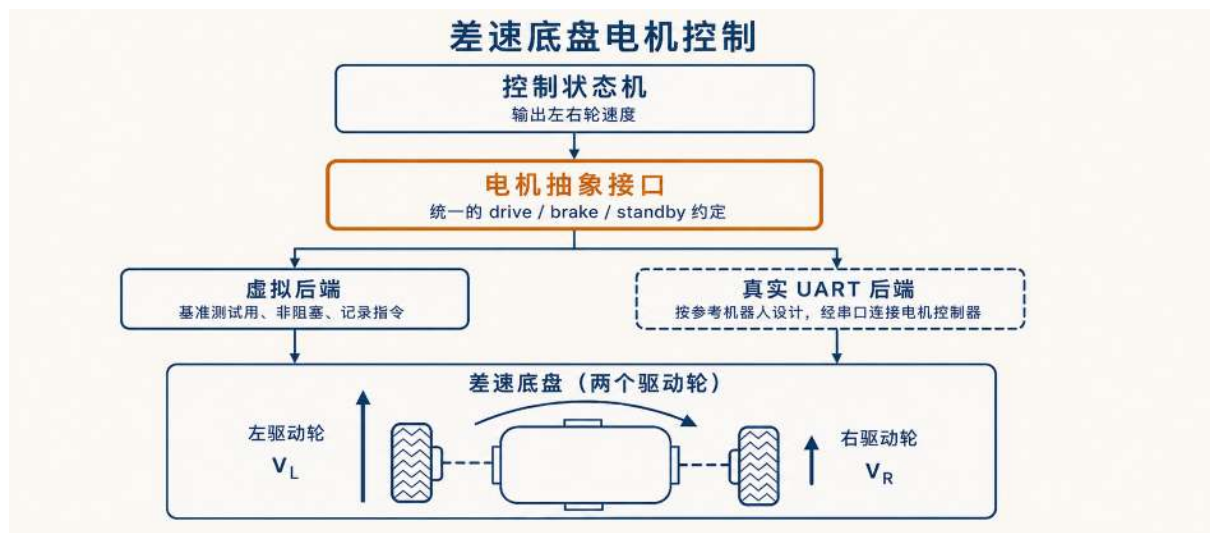


图 10 从控制状态机到 H 桥的电机控制链路，左右轮速经电机抽象层映射为四路 PWM 占空比

控制环的核心是一台状态机，它在追球、抓取、找桶、接近、投放几个状态之间流转，每一帧都输出一份带着动作类型与左右轮速的指令。追球时它把球心相对画面中线的偏移作为转向偏置，偏移越大偏置越强，远处采用两轮同向的差速行进，把偏置一加一减地叠加到一个随目标

面积递减的基础速度上，越近基础速度越低；当球已足够近却尚未对正抓爪时改为两轮反向的原地转身，使目标始终留在视野内；一旦球进入设定的近距并落在抓爪正前方的窗口内连续若干帧，便切到制动并触发抓取。接近桶的逻辑同出一辙，区别只在阈值与增益。这样状态机吐出的始终是一对范围在 $[-100, 100]$ 的有符号轮速，正值代表该侧向前。

状态机之上是一层电机抽象，它把控制语义与具体硬件隔开。这层接口只有三个动作，**drive** 按左右两路设定轮速，**brake** 主动锁住两轮，**standby** 让输出断电滑行。控制环只与这三个动作打交道，并在这一层做死区补偿，任何非零但过小的指令都会被抬到一个最小速度之上，否则电机只发热而不转动。再往下，这一对有符号轮速被翻译成驱动 H 桥所需的占空比，电机随之在底盘上实际转起来，机器人完成对球的追逐、抓取与投放。

从轮速到 H 桥的四路 PWM 两个直流电机经一块 DRV8833 一类的 H 桥驱动板供电。H 桥用一对开关的通断决定电流方向，从而决定电机正转、反转或停止，每个电机需要两条方向线，两个电机合起来就是四路带占空比调节的 PWM。占空比的高低决定有效电压，也就决定了转速，而那对有符号轮速的符号则选定每个电机两条线中哪一条出 PWM、哪一条拉低，由此定下转向。一条轮速为正就让正转线按其大小出 PWM、反转线拉低，为负则两条线互换，为零则两条线同时拉低让电机自由滑行；制动则把两条线同时拉高，让电机绕组短接而锁死。四路占空比按这一规则从左右轮速算出，差速底盘的全部行为都落在这四个数上。

在 Starry 上提供与 Linux 一致的 PWM 通路 为了让上面这条链路在 Starry 与 Linux 上以同一份代码运行，Starry 提供了与 Linux 兼容的 `/sys/class/pwm sysfs` 接口，应用通过导出通道、写入周期与占空比、再启停通道来操控每一路 PWM，无需关心底层寄存器。在 RK3588 平台一侧，我们接好了对应的时钟、引脚复用与寄存器配置，把 OrangePi 5 Plus 四十针排针上引出的 PWM 通道接到这套 sysfs 之下，于是同样的导出、设周期与占空比、启停的操作序列在两个系统上行为一致，电机抽象层写出的四路占空比可以原样落到硬件上。**该 PWM 驱动正在向上游提交。**

电机抽象与底层 PWM 之间是按值传递的轮速与占空比，而非跨越内核的复杂调用，因此从相机一帧到落到电机上的那条指令始终走在纯计算与一次 sysfs 写入的轻量路径上，不会被驱动阻塞拖慢，与本报告其余环节追求的帧到指令低时延保持一致。

5.2 机械臂控制

抓取这一步与底盘行进完全不同。底盘只需把两个轮速持续刷新给电机，机械臂却要把多个关节按先后次序摆到一组确定的角度，再让末端的夹爪在合适的时机合拢或张开。这类动作天然由一块专用的运动控制器在臂体本地闭环执行，它读取各关节的当前状态，按指令把关节驱到目标位姿并维持，主控只负责下达“去哪个位姿”这样的高层意图。主控与这块控制器之间因此需要一条串行链路来往返传递状态与指令，而 RK3588 主板对外并没有现成的串口引脚直连到臂上，机械臂一侧给出的是一个 USB 接口。中间的桥接由一颗 CP210x USB 转串口芯片承担，它在 USB 与异步串口之间做协议转换，在系统里以一个串口设备的形态呈现，应用打开 `/dev/ttyUSB0` 即可像操作普通串口那样与臂上的运动控制器通信，读取臂的状态并发出定位与抓放指令。

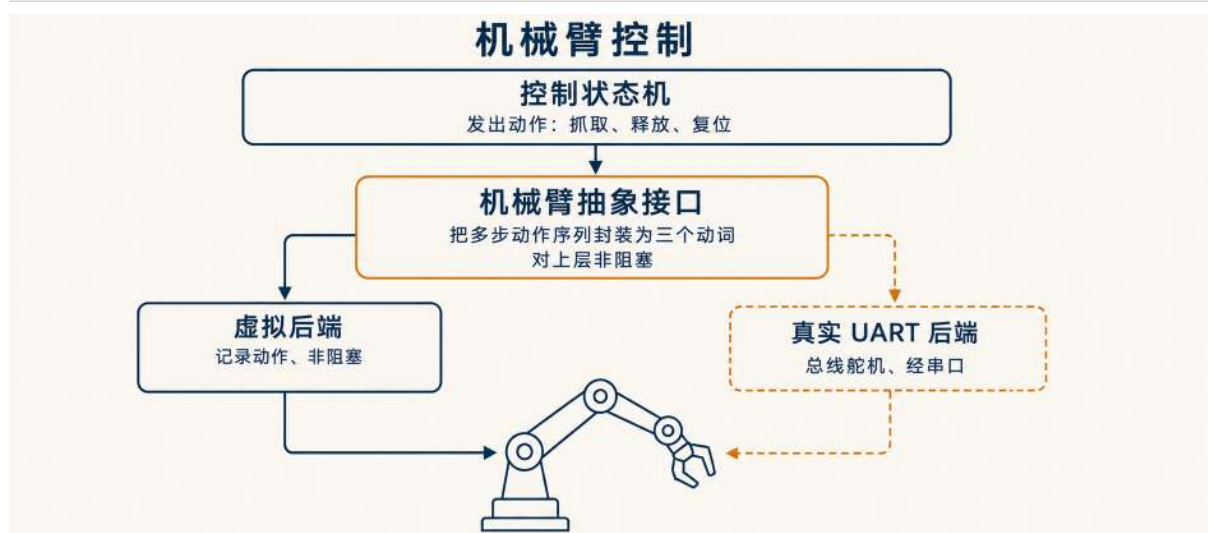


图 11 多关节机械臂与夹爪，经 CP210x USB 转串口桥接挂到 RK3588 主板的 /dev/ttyUSB0

Starry 起初并没有这条 USB 串口通路。USB 主机栈枚举到 CP210x 设备后，并不会自动把它变成一个可供用户态读写的串口节点，缺的是一层把 CP210x 的厂商控制传输翻译为串口语义、并把它登记为 /dev/ttyUSB0 的设备驱动。本队为此在内核里补齐了 USB 串口 tty 支持，其中的 CP210x 后端对齐 Linux 同名驱动的行为，完成设备的初始化序列、波特率配置与控制传输处理，使枚举到的 CP210x 在 Starry 上呈现为标准的 /dev/ttyUSB0。这部分工作以合并请求 #1378 提交至上游并已合并，附带的主机端单元测试覆盖了初始化与配置路径，因此这条通路不是临时打通的，而是按上游可维护的形态落地的。

把多步位姿收敛为几个高层动作 控制回路按帧产出决策，它没有余裕去逐关节地编排运动。把一次完整抓取展开来看，臂要先回到一个张开的待命位姿，发现球后伸向目标、合拢夹爪、抬起，找到回收筐后再在筐口上方张开夹爪松手，每一步都是若干关节角的有序变化。直接把这套序列摊在控制回路里，回路就会被关节级的细节淹没，也会在等待臂体到位时阻塞，错过新到的帧。机械臂的后端抽象因此把这些多步序列归并为面向回路的几个高层动词，分别是回到待命位姿、抓取、以及松手投放。待命动词把臂带回原点并张开夹爪；抓取动词把伸向目标、合拢与抬起这一连串动作合为一体；松手动词在筐口上方张开夹爪完成投放。回路只需在状态机给出抓或放的判定时下达对应的一个动词，余下的关节编排沿串口链路在臂上自行展开。

与控制回路的衔接 这几个动词向控制回路呈现为非阻塞的调用，状态机每帧至多触发其一。控制器在收到一帧检测结果并跑完状态机后，只在动作判定不为空时才按判定选择待命、抓取或松手中的一个下达，随即继续处理下一帧，而不在调用点上等待臂体到位。这样抓放这类耗时较长的物理动作就不会顶在帧到指令的关键路径上，感知与控制得以维持稳定的帧率，机械臂则沿 /dev/ttyUSB0 这条串口链路把收到的高层动词落实为对实物臂的关节定位与夹爪开合，真实地完成对网球的抓取与在回收筐上方的投放。

5.3 拾球控制状态机

机器人要把场上的网球逐个拾起再倒进收集桶，这是一连串有先后次序的动作，而不是一个可以一次算完的函数。它得先找到球、追上去、把球夹住，再转身找到桶、靠近桶、把球松开，然后回到找球。每一步成立的前提都依赖上一步的结果，且依赖此刻相机看到了什么。把这套行为组织成一台有限状态机，机器人在任一时刻只处于一个明确的阶段，每帧依据当前阶段和最新的一次检测决定下一步怎么走，再在条件满足时切换到下一个阶段。

整套流程由 `GameState` 枚举的五个状态构成，分别是追球 (`CHASE_BALL`)、夹取 (`GRAB`)、寻桶 (`FIND_BUCKET`)、近桶 (`APPROACH_BUCKET`) 与投放 (`DEPOSIT`)，机器人启动即进入追球。状态机自身不做任何输入输出，也不睡眠等待，它只接收一个 `Detection` 并返回一个 `ControlOutput`，后者描述这一帧底盘该如何动（前进、刹车或待机，附带左右轮速度）以及机械臂该做什么动作。这样一帧从相机到指令的时延就等于纯计算时延，不会被状态机内部的忙等拖长。这一点与参考实现不同，参考实现把夹取与投放写成在循环里忙等数百毫秒的阻塞段，本实现把夹取与投放也建模成短暂的、非阻塞的显式状态，靠帧计数自然退出。

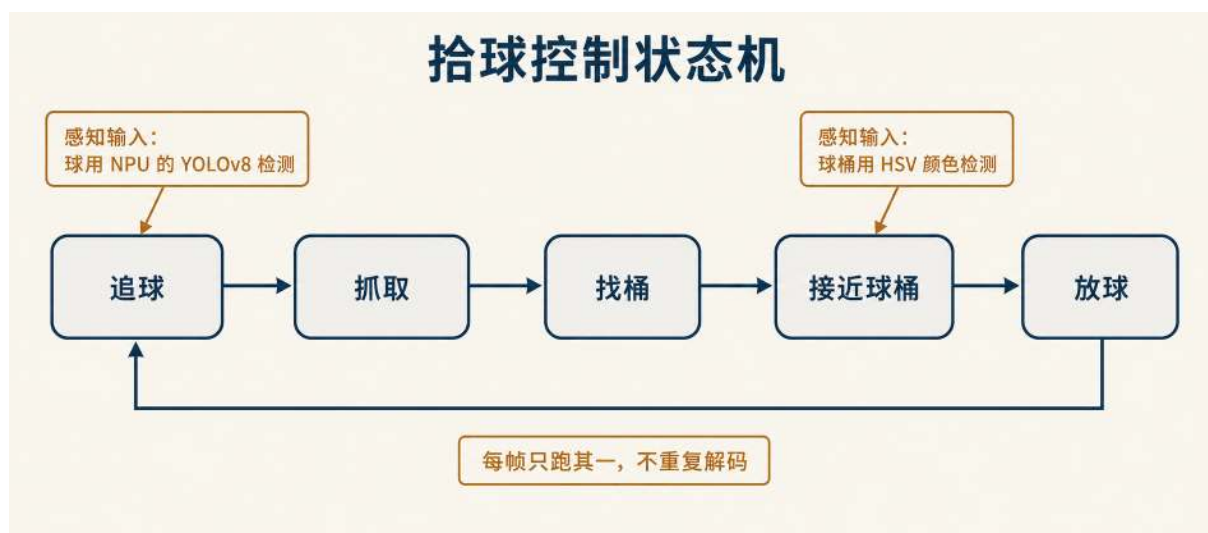


图 12 拾球控制状态机的五个状态与触发各次切换的条件，回路在投放完成后回到追球，开始下一轮

状态之间的流转 追球状态把球的检测结果转成对底盘的转向与前进量。球的水平偏移决定转向偏置，球在画面里占的面积比 (`area_ratio`) 作为距离的代理量决定前进速度，远则全速、近则收速。当球足够近且持续若干帧 (`stop_confirm_cnt`) 稳定地对准夹爪的目标位置时，状态机刹车并切入夹取。夹爪并不在光轴正中，而是装在偏右的位置，因此对准的目标是一个带偏置的点 (`stop_center_offset`) 而非画面中心。若某一帧没看到球，机器人先朝球最后出现的方向转动搜寻，搜寻若干帧仍未找回，便改为左右往复扫视，扫视方向每隔固定帧数翻转一次。进入夹取后，机械臂的夹取动作只在该状态的第一帧下发一次，再经若干帧 (`grab_settle_frames`) 安定后转入寻桶。寻桶时底盘原地旋转，一旦连续若干帧 (`bucket_confirm_cnt`) 都看到桶，便切入近桶；近桶时按桶的偏移转向、按桶的面积比收速逼近，中途若桶丢失超过若干帧 (`bucket_lost_frames`) 则退回寻桶，桶在画面里占满到投放阈值 (`bucket_area_deposit`) 时刹车并切入投放。投放时机械臂的松开动作同样只在第一帧下发，安定若干帧 (`deposit_settle_frames`) 后机械臂复位，

状态机回到追球，一轮完整的拾球至此闭合。

一帧只跑一个检测器 两路感知喂给状态机，球由运行在 NPU 上的 YOLOv8 检测（（四）），桶则由色调饱和度明度（HSV）颜色检测识别。读 `BucketDetector` 可以确认桶的识别全程不碰神经网络，它把 RGB888 转为 HSV，用一个跨越色环两端的双区间红色掩膜，再以两遍并查集连通域标记取出像素数最大且超过最小面积阈值的红色块，把它的中心与面积比作为桶的观测。两个检测器的代价并不相同，前者要占用加速器跑一次推理，后者只在 CPU 上扫一遍像素，因此让两者每帧都跑会平白浪费一次推理。状态机据此对外暴露一个 `perception_mode()`，在追球与夹取两个状态下它返回 `BALL`，在其余状态下返回 `BUCKET`，感知线程依据这个值决定下一帧只运行哪一个检测器。这样同一帧相机图像永远只被一路处理，绝不会为了同时拿到球与桶而把同一帧解码或处理两遍，这一点与参考实现在寻桶相关状态下把同一张 JPEG 解码两次的做法正相反。

把检测结果转为动作 状态机消费的是最新的一帧相机图像（（一）），输出的则是底盘指令与机械臂动词。控制器（`Controller`）拿到 `ControlOutput` 后，把 `Drive` 译为对左右轮速度的设置、把 `Brake` 与 `Standby` 译为刹车与待机，交给差速底盘执行（（一）），把 `Grab`、`Release`、`Ready` 三个机械臂动词交给机械臂执行（（二））。为避免在热路径上反复下发完全相同的电机指令，控制器只在指令相较上一帧发生变化时才真正写下去。至此，相机的一帧经感知、状态机决策、再到底盘与机械臂，构成一条从感知到控制的闭合回路，机器人就在这条回路上一颗接一颗地把球拾起。

6 支撑机器人运行的系统能力

把机器人程序在 Starry 上从无到有地跑起来，除了视觉与本体的各部件驱动之外，还需要一连串底层的系统能力。从开发板上电到挂载根文件系统，从线程在大小核上的放置，到可测量优化效果的性能剖析，再到与机械臂控制器通信和在两套系统间快速切换，这些能力共同构成了机器人得以稳定运行与持续迭代的地基。

6.1 启动与根文件系统

机器人上的所有软件都来自一张 SD 卡。这块板子上电后，固化在芯片里的一小段引导程序先把 U-Boot 从卡上装入并运行，U-Boot 再把内核镜像读进内存并交出控制权。内核拿到控制权后做的头等大事，是把 SD 卡上那块以 ext4 格式存放的根文件系统挂载起来，应用程序、模型权重与配置才有处可寻。Starry 在这里与 Linux 走同一条路，它要挂载的正是 Linux 也在用的那一块 ext4 根分区，存储介质、分区布局与文件系统格式都不变，变的只是挂载它的内核。

SD 卡这类块设备并不能像内存那样随取随到，向它发起一次读取要经过排队、提交、等待硬件搬运、最后收取完成信号这样几步。承担这套时序的是板上的 dwmmc 控制器，也就是新思科技（DesignWare，一家提供可复用硬件电路设计的厂商）那套被广泛集成进各家芯片的 SD/eMMC 主机控制器。一次数据传输完成后，控制器本应拉高一根中断线，通知内核这笔请

求已经办妥；内核据此把等待这笔请求的任务唤醒，去取回数据。这种由硬件主动报告完成、内核被动等待的方式称为中断驱动，它让等待中的核心可以彻底休眠而不必空转轮询，是块设备完成路径的常规做法。Starry 的块运行时正是按中断驱动来等待每一笔传输的完成。

把这块 SD 卡注册为块设备之后，启动却停在了挂载这一步。块运行时提交了第一笔 ext4 读取，随即进入中断驱动的等待，可那根完成中断始终没有到来，等待便永远不会结束，挂载就此挂死。逐层排查后确认问题出在控制器一侧。中断的路由与使能都核对无误，从设备树到中断控制器（GIC）的这条线接得没错，控制器寄存器里的中断使能位也确实置上了，唯独这片 RK3588 上的 dwmmc 控制器在一次数据传输结束后并不可靠地拉高它的完成中断。信号根本没有发出，内核这一侧再正确也无从被唤醒，第一笔 ext4 读取于是永远不返回。

棘手之处在于，控制器是否真把中断送达，没有任何一个内核能直接读到的状态位能回答。运行时能查询的 `is_irq_enabled()` 反映的只是控制器寄存器里那一位使能位，它说的是“我已被允许发中断”，而非“中断确实经 GIC 送到了处理函数”。这两者在正常硬件上等价，在这片控制器上却分了岔。既然没有直接的判据，唯一能观测到这一故障的地方，就是等待本身等了太久。

据此在完成路径上补了一道对中断是否可用的判断。中断驱动的等待不再无限期睡下去，而是带一个有上限的期限。这条期限通过 `task_wait_until_timeout` 实现，任务睡到完成条件成立即被唤醒，或者睡满期限后自行醒来；它把底层带超时的等待原语的返回值取反，使得返回为真当且仅当真正观测到了完成。期限取两秒，这是个有意放宽的值，普通慢速设备远不至于跑满，唯有完成信号确实缺席才会触到。一旦在两秒内没有等到中断，运行时便判定这台设备此刻的完成中断不可用，先转为轮询去亲自确认这笔传输究竟办妥没有，并把这台设备就地切到轮询，往后它的每一笔传输都直接读硬件状态而不再等中断。这个切换是按设备一次性锁定的，由设备上一个记录完成方式的状态位 `completion_mode` 经 `demote_to_polling` 翻成轮询，此后不再翻回，免得每一笔传输都白白赔上两秒的期限。中断正常的设备完全不受影响，它们的完成中断照常即时唤醒等待者，期限根本不会触发，也就没有任何额外开销。

```
// 蝻激簪颢擦读筌史洸纡孽溜胃沫猖溜礅睹礅
let completed = task_wait_until_timeout(
    || keys.iter().any(|&k| self.pending.lock().result(k).is_some()),
    IRQ_COMPLETION_FALLBACK_TIMEOUT, // 睹礅
);
if !completed {
    // 筌史耘蝻激脖洸擦窄湊弹繇辣胃沫蝻激汕痛菽孽
    // 曹纂簪洸碇苈孽般礅礅殒浞燃澈窖纡睹礅
    self.demote_to_polling();
    return self.polling_wait_for_any_key(&keys);
}
```

Listing 1 完成等待的超时回退（节选自 `block_runtime`）

这道回退同时落在三条等待路径上，单笔请求的完成等待、队列推进的等待，以及面向多笔在途请求的等待，任意一条等满两秒都会把设备切到轮询，缺席的中断因此不再能拖住挂载。设备出厂即以轮询方式登记，只有在中断处理函数成功注册之后才被提升为中断驱动，这道回退

正是这条提升的对称反向，把一台被误判为中断可用的设备拨回它本就稳妥的轮询路径。

需要如实说明的是，这只是一道变通，不是根因修复。它保证的仅仅是挂载稳定可达，至于这片 dwmmc 控制器为何在传输结束后不拉高完成中断，仍是悬而未决的问题，根子在控制器一侧而非内核的中断处理。真正的修复应当落在 dwmmc 主机驱动对控制器中断使能与中断屏蔽寄存器的设置上，让控制器如约拉高那根线。在那之前，轮询回退让根文件系统能稳定挂上，整套系统得以启动，后续各部件才有立足之地。**这项工作待提交至上游（提交 5c36d90e3）。**

6.2 大小核调度与负载均衡

RK3588 不是一块由八个相同核心拼成的处理器，它把两种规格的核混在同一块芯片上，四个 Cortex-A55 小核运行在 1.8 GHz，四个 Cortex-A76 大核运行在 2.256 GHz。大核的微架构更宽、流水线更深，同样一段代码在大核上跑得明显比在小核上快。这种异构布局通常称作大小核 (big.LITTLE)，它的好处是让轻负载落在省电的小核而把重活交给大核，可一旦操作系统不区分这两类核，把一个计算密集的线程随手放到一个小核上，它就只能以小核的速度慢慢跑，旁边空着的大核帮不上忙。对一条每帧都要在限定时间内跑完采集、解码、缩放与推理的感知流水线来说，线程被放到哪个核上，直接决定了这一帧能否按时交付。

Starry 继承自 ArceOS 的任务调度在这一点上采取的是最朴素的策略。一个线程被创建或被唤醒时，调度器默认把它放到当前正在执行放置动作的那个核上，借此让它的缓存保持温热，只有当线程的亲性和掩码不允许留在本核时，才退而用一个简单的轮询计数去挑下一个允许的核。这条路径在 `select_run_queue` 里就是先看 `task.cpumask().get(current_cpu)` 是否成立，成立就直接落在本核，整个过程不读取任何一个核当前有多忙，也不区分这个核是大核还是小核。机器人程序起初没有设置任何 CPU 亲和性，于是采集、解码、缩放与推理这几个本可以分散开的线程全都被放到了最先启动它们的那个 A55 小核上，在 `cpu0` 上首尾相接地串行执行，彼此争抢同一个最慢的核。这正是迁移到 Starry 之初端到端帧时间偏高的根源，后续把推理线程手动钉到一个 A76 大核上得到的数倍提速，以及那一组随核位变化的实测数字，都记在性能优化一节（七）里。

让调度器区分大核与小核 针对这一缺口，本队做了一个对大小核有感知的负载均衡器原型，它不替换原有的调度算法，而是在放置决策与空闲核两处补上以核能力为权重的均衡，并以一个默认关闭的 `sched-loadbalance` 特性开关控制，开关不打开时默认构建与 QEMU 持续集成的产物逐字节不变。它要回答的第一个问题是每个核到底有多强。这一信息来自设备树，`cpu_capacities` 在系统启动时遍历设备树里的各个 `cpu@*` 节点，优先读取节点上的 `capacity-dmips-mhz` 属性，读不到时再退而看节点的 `compatible` 字符串，认出 `arm,cortex-a55` 就记为 530，认出 `arm,cortex-a76` 就记为 1024，两者都没有时给一个 1024 的默认值，结果按逻辑核号缓存下来。这样大核与小核之间约一比二的算力差距，就以一个归一化的能力权重被调度器读到，而在所有核都相同的同构板子或 QEMU 上，这套权重退化为全相等，均衡器随之退化为单纯的负载分摊。

按能力加权的负载去挑核 有了每个核的能力，放置决策就不再盯着当前核，而是去找加权后最轻的那个核。衡量轻重的不是裸的就绪任务数，而是把它除以核能力再乘以一个固定基准得到的有效负载，即 $\text{load} * 1024 / \text{capacity}$ 。同样一份待跑的工作放在小核上要花更久，除以更小的能力之后它的有效负载就更高，于是 `select_least_loaded` 在遍历亲和性允许且已经上线的各个核时，会自然地先把计算往大核上堆，等大核被填满、它们的有效负载追上小核了，工作才开始往小核溢出。挑核时对线程上一次所在的核留了一点偏向，把它的负载当作低一档来比较，使在负载相当的情况下线程尽量回到缓存温热的那个核。线程被唤醒时走的也是同一套加权择核，只是把上次所在的核作为这点偏向的对象。除了放置这一头，空闲的核也不再立刻停下来等中断，而是在 `idle_pull_once` 里扫一遍其它在线的核，从加权负载最重的那个核上偷一个既符合目标核亲和性、又没有正在别处运行、也不是空闲任务的就绪线程过来自己跑；定时器节拍上还有一个限频且带迟滞的反向推送，本核明显比最轻的远端核更忙时主动推一个线程过去，避免只靠偷取在某些情形下反应不及。这些跨核搬运都严格遵守同一时刻至多持有一把调度器锁的纪律，被搬运的线程在两个队列之间的空档里只由一个本地引用持有，因而既不会丢失也不会被两个核同时运行。

原型的定位与局限 这个均衡器在 QEMU 的多核环境里通过了一组覆盖亲和性迁移、抢占与并发竞争的测试，相关的就绪队列计数与可窃取任务的接口也都各自带了单元测试。但必须如实说明，它目前仍是一个原型，是日后把调度真正做成对体系结构有感知的基础，而不是已经接管生产负载的调度器。对机器人当前这套以手动钉核运行的单线程化负载而言，工作早已被显式地固定在选定的核上，再叠一层自动均衡并不改变结果，因此机器人实际运行时仍沿用手动钉核，把这个原型发展为完整的体系结构感知调度留作后续工作。它要解决的真问题也正在这里，跨核唤醒的延迟仍有长尾，最差的一帧约 487ms，而 Linux 同条件下约 22ms，伴随约一成的丢帧，把这条尾巴收拢到与放置同样可控的程度，是这块调度工作尚未走完的部分。这一原型尚未提交至上游（待提交）。

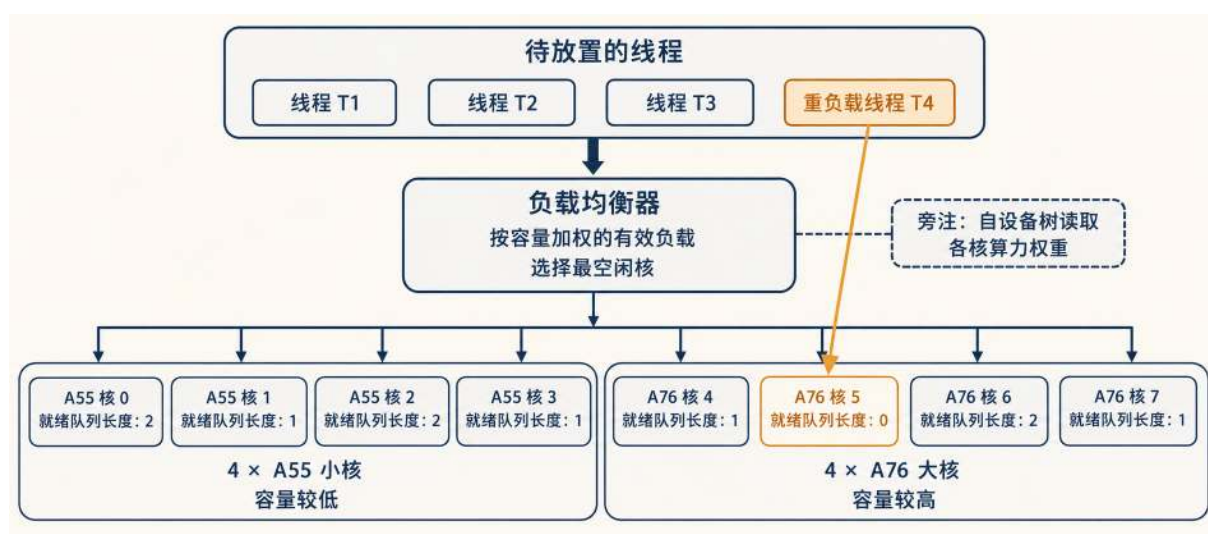


图 13 大小核异构下的放置：能力权重自设备树读出，放置与空闲偷取都以 $\text{load} * 1024 / \text{capacity}$ 的有效负载择核，计算先堆向 A76 大核，填满后才向 A55 小核溢出

6.3 基于硬件 PMU 的 perf

要判断一段推理究竟慢在哪里,仅靠墙钟时间是不够的。墙钟只告诉我们一步花了多久,却说不清这段时间里处理器到底在算还是在等。每一颗现代处理器内部都埋着一组性能监控部件(PMU,处理器内部的事件计数器),它在不打扰程序运行的前提下,按硬件事件累加计数,比如经过的时钟周期、退休的指令条数、各级缓存的访问与缺失、分支的执行与预测失败。RK3588 的 A55 与 A76 两种核都实现了 ARMv8 体系的第三代监控规范 PMUv3,对外是一个专用的 64 位周期计数器 PMCCNTR_ELO 与若干个可编程的 32 位事件计数器 PMEVCNTRn_ELO,每个可编程计数器配一个事件选择寄存器 PMEVTYPERn_ELO 来指定它数哪一种事件,整组计数器再由 PMCR_ELO 统一开关。Linux 上的 perf 工具正是读这些计数器来给一段程序画像,把它的执行落到周期与指令这一层去解释。

Starry 此前只有软件与追踪类的事件,没有触达这组硬件计数器的通路。本队为它补上了一个原生的 perf,对齐 Linux perf 的语义,覆盖计数(perf stat)、采样(perf record)与符号化报告(perf report)三种用法,使既有的 perf 二进制无需改动即可在 Starry 上对 RK3588 取数。这套能力本身是操作系统的一项通用设施,同时又是本项目优化所依赖的量具,第(一)节的测量方法与第七节的优化都建立在它读出的数字之上。它最初只能为单个任务在单核上计数,随后扩展为对称多处理下的逐核采样,最终又针对 RK3588 的大小核异构结构,按集群分别配置各自的 PMU,并补上必要的计数器复用与逐核分发。这一系列工作以合并请求 #1395 提交至上游并已合并。

把事件落到正确的 PMU 寄存器 计数从识别事件开始。perf 递交的 perf_event_attr 既可能用 Linux 抽象的硬件事件号(如周期、指令、缓存),也可能直接给出处理器原生的 ARM 事件号。前者须翻译为后者再写入 PMEVTYPERn_ELO,例如周期事件对应 ARM 事件号 0x11。周期事件若不带采样,则直接占用专用的 PMCCNTR_ELO,其余事件各取一个可编程计数器。一个看似简单却容易错的事件是分支指令计数,它在 A55 与 A76 上的最佳近似并不相同,A55 倾向于用 PC_WRITE_RETIRED (0x0C),A76 则用 BR_RETIRED (0x21)。本实现据当前核所通告的事件支持位逐核裁决,而非全局写死一个号,否则在小核上数到的会是另一回事。设备本身经 sysfs 暴露为 armv8_pmu_v3_0,perf 据此发现这块 PMU 并选取事件。

从单核计数走到逐核分发 perf stat -a 要对整机每一颗核分别取数,而发起调用的线程未必就跑在目标核上。计数器是处理器核私有的,PMEVCNTRn_ELO 只能在它所属的那颗核上配置与读取,跨核去碰别人的计数器毫无意义。为此每颗核维护自己的一组计数器池,对某一目标核的配置、启动、读取、停止与释放,若发起者恰在该核便就地执行,否则经一次同步的处理器间中断投递到目标核去做,发起线程阻塞到它返回,这与 Linux 用 smp_call_function_single 把操作搬到目标核如出一辙。计数器的全局开关 PMCR_ELO.E 也因此变成逐核各自置位。

计数器不够用时的轮换 可编程计数器的数目有限,当一个任务同时打开的事件多过本核的计数器数目时,它们无法同时上硬件。此时各事件轮流占用计数器,被注册到周期性的调度器节拍上,节拍每到一次就把当前任务的事件向前轮换一格,于是每个事件都能在一段时间里被采到。

轮换的代价是某个事件并非全程在数，实现因此分别记录每个事件处于使能状态的时长与真正持有硬件计数器的时长，perf 拿到这两个时长后按使能比真实的比例把读数放大回去，这正是 Linux 计数器复用的做法。采样路径则另有讲究，一个带采样周期的事件必定占用一个可编程计数器（即便是周期事件也不再走专用计数器），并在计数溢出时触发中断，由溢出处理程序记一笔采样，处理程序运行在中断上下文里，不分配内存也不取可睡眠的锁。

在两类核上分别取数 RK3588 是大小核异构的，同一段推理放在 A55 与放在 A76 上，其周期与指令的关系并不一样，用一块全局 PMU 去笼统地数会把两类核的行为混在一起。本实现据 MIDR_EL1 的实现者与部件号把每颗核归类到所属集群，实现者为 Arm (0x41)、部件号 0xD05 即 Cortex-A55、0xD0B 即 Cortex-A76，再把两个集群各自作为一块独立的 PMU 经 sysfs 暴露为 armv8_cortex_a55 与 armv8_cortex_a76。开在某一集群 PMU 上的事件只在该集群的核上配置与计数，于是同一段负载在小核与大核上的画像可以分开取得并直接比较。

在本项目中读出的结论 这套量具在本项目里有三种用法。其一，把周期与指令一起读出得到每周指令数（IPC），再配上缓存事件，用以判断一段负载是受算力约束还是受访存约束。其二，把流水线每一级的开销从墙钟挪到周期与指令这一尺度上，看清时间究竟耗在何处。其三，在两个集群上分别读计数，量化同一段推理在 A55 与 A76 上的差异。借由它，本队得以确认推理是受算力约束的，IPC 偏高而缓存缺失率极低，约 2.3%，并非卡在访存，这一事实是把推理放置到大核的直接依据，相关分析见第七节。

6.4 USB 串口与 reboot 系统调用

机械臂的运动控制器并不挂在开发板的板载串口上，而是经一颗 CP210x USB 转串口桥接芯片接入，在系统里表现为 /dev/ttyUSB0 这样一个普通的终端设备。机器人通过这个节点读取机械臂的当前状态，并把抓取与归位指令写回去（见（二））。USB 转串口桥的作用是把一段异步串行的 UART 字节流封装进 USB 的批量传输，主机一侧并不存在真正的 UART 控制器，波特率、数据位、停止位这些线路参数都不是去写某个串口寄存器，而是经 USB 的控制传输下发给桥接芯片，由它在另一侧还原成对应的电平时序。因此驱动要做的事分成两层，一层是把芯片配置成所需的线路模式，另一层是把应用读写的字节与芯片的批量端点对接，并让这一切套进内核既有的终端设备模型。

从描述符识别芯片并定位数据端点 一个 USB 设备插入后，主机先取回它的设备描述符与配置描述符。本队的可复用驱动核心是一个 #![no_std] 的库，它直接在这份原始描述符字节流上工作。识别 CP210x 的依据是设备描述符里的厂商号 0x10c4 与产品号 0xea60，二者同时匹配才认定。随后它遍历配置描述符下的各个接口，挑出那个厂商自定义类（接口类 0xff、子类与协议均为零、且为零号备用设置）的数据接口，再在该接口下按方向各取一个批量端点，得到一进一出的端点地址对。描述符的解析全程做长度与类型校验，遇到越界或长度为零的项即停止，避免在一份畸形的描述符上读出错位的字段。这一层只产出一个 UsbSerialPort，记录接口号与批量端点地址，不触碰任何具体的传输提交方式。

按厂商约定配置串口参数 CP210x 的所有配置都走 USB 控制传输里的厂商请求，请求类型固定为厂商方向接口 (0x40 | 0x01)，请求号则对应芯片手册里的各项操作。打开一条串口时，驱动依次发出启用 UART (IFC_ENABLE, 值 0x0001)、设置波特率 (SET_BAUDRATE, 把波特率以小端四字节作为数据载荷)、设置线路控制为 8 数据位 (SET_LINE_CTL, 值 0x0800)、拉起 DTR 与 RTS 并置其写使能位 (SET_MHS)，最后下发一段全零的流控配置 (SET_FLOW) 以关闭硬件流控。这套时序与 Linux 上的 cp210x 驱动一致，目的就是把芯片置于一条不带流控的裸 8N1 链路，让上层只看到透明的字节流。波特率不是写死的常量，而是从终端的 `termios` 里取，应用通过标准的终端接口改波特率时，驱动会重新下发一次 SET_BAUDRATE，机械臂控制器所需的速率因此由用户态决定而非驱动假定。

把字节流接进终端设备 内核侧的接入复用了已有的 `Tty` 框架，与板载串口共用同一套行规与终端语义，区别只在底层的收发后端。`/dev/ttyUSB0` 是一个静态存在的设备节点，打开时才惰性地占用第一个被识别到的 USB 串口适配器，占用一个 `usbfs` 会话并认领对应接口，再跑前述的初始化时序，使得终端可以先于适配器插入就存在。会话建立后，一个接收工作线程持续在批量输入端点上做读入，把收到的字节推进接收队列并唤醒等待的读者；发送侧则把应用写入与终端回显的字节并入同一条发送队列，由发送路径在批量输出端点上排空，两路输出共用一把输出锁以保证字节顺序不被打乱。当前的 xHCI 后端尚未提供基于 Stop Endpoint 的传输取消能力，因此一条仍在阻塞读的接收传输不会被强行打断，会话的拆除被推迟到接收线程观察到一次真实的 USB 完成或设备错误时再收尾，这是当前后端能力下的稳妥处理，而非可随时中断的理想形态。该驱动以合并请求 [#1378](#) 提交至上游并已合并，host 侧单元测试覆盖了厂商号匹配、非 CP210x 设备的拒识、批量端点配对以及控制时序的逐条核对，并在 OrangePi 5 Plus 上完成了实机验证。

一条命令切换整套系统 开发期间只有一块远端开发板，而验证又要求在同一块板上反复切换 Linux 与 Starry，跑同一个二进制做对照。若每次切换都靠人工断电再上电，节奏会被这一步拖住。为此本队实现了 `reboot` 系统调用，切换从一次手动电源循环变成一条命令，迭代因此明显加快。这个调用同样服务于面向人工智能开发的那套自动化回路（见 九），让镜像替换后的重启不再需要有人守在板边。

实现遵循 Linux 的 `reboot` 语义。入口先校验调用者持有 `CAP_SYS_BOOT` 能力，再核对两个魔数，第一魔数须为 `0xf00dead`，第二魔数须落在 Linux 认可的几个取值之内，任一不符即以 `EINVAL` 拒绝，避免误触的写入意外重启整机。命令字区分几种行为，`RESTART` 与 `RESTART2` 触发重启，`HALT` 与 `POWER_OFF` 触发关机，`CAD_ON` 与 `CAD_OFF` 则直接返回成功而不做实际动作。重启与关机之前都会先调用一次文件系统的下刷与卸载，让缓存中的数据落盘，再把控制交给平台电源层。`RESTART2` 携带的重启原因字符串当前被忽略，`CAD_ON` / `CAD_OFF` 也尚未真正改变组合键的行为，这两点已如实记录为已知限制。

重启最终如何作用于硬件 平台电源层把重启统一抽象为一个 `reset()` 操作，具体动作随体系结构而异，在本机所用的 aarch64 上走 PSCI（电源状态协调接口，由更高特权级的固件提供

的标准电源服务)。系统启动时, 电源模块从设备树里找到 `arm,psci-1.0` (含 0.2 与无版本号的回退) 节点, 读出其 `method` 属性以确定调用固件的方式是 `smc` 还是 `hvc`, 并把它记下。真正重启时, 据这一方式发起 PSCI 的 `SYSTEM_RESET`, 由固件接管整机复位; 调用经 `smccc` 库封装, 若固件返回错误则打印诊断并停在等待中断的空转循环里。关机走的是同一条路径下的 `SYSTEM_OFF`。该系统调用以合并请求 [#1358](#) 提交至上游并已合并。

7 性能优化

推理链路打通、各部件就位之后, 工作转入以 Linux 为基线的性能优化, 对应评审的第三阶段。本节先建立可测量的剖析口径与工具, 再给出两侧的性能概览与差距来源, 随后分别从用户态与内核侧展开优化。

7.1 剖析方法与测量工具

可靠的测量是优化的前提。已有的剖析仅提供各阶段的平均值, 既无尾部分布, 也无内核侧观测与受控消融, 其结论停留在某阶段慢若干倍的层面, 难以定位开销的物理成因。为此, 我们在同一份可执行文件与内核中补齐了观测能力。

应用层在每次推理上记录由相机帧到达主机至检测结果就绪的全链路时延, 并给出 p50、p95、p99、p99.9 与 max 各分位, 同时以逐帧 CSV 导出供离线分析。冷启动路径单独记录, 将从进程启动到首次推理完成的时间拆分为模型初始化、采集初始化、首帧等待与首次推理四段。我们还把拷贝路径与零拷贝路径以同一前端编译为两个二进制, 构成输入输出方式的 A/B 对照; 所谓零拷贝, 是让一段缓冲在各阶段之间不再被复制, 而由硬件直接写入下游缓冲。NPU 内部的逐算子耗时以一次性诊断的方式采集, 默认关闭, 以免扰动 `run` 的主分位 (p50、p95 等) 数据。内核侧则加入一个可选的计时特性, 以无锁原子计数聚合与 NPU 相关 `ioctl` 各阶段的调用次数与耗时, 将原本只能推测的缓存同步与提交开销转化为可测量的内核耗时。

为在 CPU 侧获得更细的归因, 我们在 Starry 上实现了一套基于硬件性能监控单元的原生 `perf` 工具, 其架构与实现见第 (三) 节。该工具在本工作中有三处用途。

- 用每周指令数 (IPC, 指单位时钟周期内执行完成的指令条数, 越高越接近算力上限, 此处并非进程间通信) 与缓存计数, 判断负载受限于计算还是访存。
- 把各阶段的开销落到周期与指令的尺度上。
- 在大小核两个簇上分别读取计数, 量化同一段推理在 A55 与 A76 上的执行差异。

为保证数据可信, 干净运行与诊断运行严格分离, 前者不开启任何会扰动主分位指标的观测。在合并上游一次较大的内核同步 (涉及调度与内存等路径) 之后, 我们对基线复测了一次, 主分位指标全部落在 $\pm 2\%$ 以内。每一项数值结论在成文之前均由多个独立流程对照原始日志逐一核对。

7.2 性能概览

Linux 基线为全篇提供对照参照。在 640×480 的实时采集下，推理绑定至 A76 大核，采样线程隔离于 A55 小核，连续运行 1792 帧无错误，端到端时延平均 25.81 ms，吞吐 29.64 fps。各阶段时延见表 3，其中 **run**，即 NPU 的一次前向推理，约占除解码外的推理各阶段耗时之和的 79% (15.4 与 19.4 ms 之比)，是单帧时延的主体。由 NPU 运行时的逐算子剖析进一步可见，该模型为卷积主导，卷积类算子占 NPU 时间的约 72%，因此只有模型层面的改动才能显著缩短 **run**。

表 3 Linux 基线各阶段时延 (640×480 实时，单位 ms)

阶段	平均	p50	p95	max
端到端	25.81	23.96	33.35	44.30
decode	5.59	4.78	10.18	—
letterbox (RGA)	1.38	1.24	2.20	2.80
inputs_set	0.26	—	0.45	7.50
run (NPU)	15.42	14.82	19.35	28.18
outputs_get	1.30	1.43	1.54	2.24
postprocess	1.06	1.19	1.25	1.86

围绕瓶颈来源的受控消融见表 4。其中以 RGA 承担缩放，相对 CPU 缩放可将该阶段由 15.82 ms 降至 1.72 ms，是基线中最显著的单项。NPU 在两核以上无法继续切分该小模型，逐算子诊断显示其负载几乎集中在一个核上，第三核没有收益；其时钟在整个运行中保持 1.0 GHz，按需调频与锁定最高频两种策略的差异落在噪声以内。此外，进程的调度时延平均仅 0.021 ms，NPU 每帧读写约 36 MB，折合约 1.06 GB/s，CPU 缓存缺失率仅 2.3%，后者由 perf 的缓存计数佐证，可见基线既不受调度饥饿约束，也不受内存带宽约束。

表 4 Linux 基线的受控消融

消融项	对照	结果
letterbox 缩放	RGA 硬件 <i>vs</i> CPU	1.72 <i>vs</i> 15.82 ms (9.2 $\times$)
NPU 核数	1 / 2 / 3 核	16.66 / 14.87 / 15.23 ms
NPU 调频	ondemand 按需 <i>vs</i> performance 最高频	19.30 <i>vs</i> 18.93 ms (噪声内)
输入输出方式	拷贝 <i>vs</i> 零拷贝	23.83 <i>vs</i> 24.65 ms

在同一份程序、同一份模型的前提下，Starry 最初配置的实时端到端时延 p50 为 202.9 ms，约为 Linux 的 7.9 倍，各阶段对照见表 5。内核计时给出了第一个判断，提交 NPU 作业的内核开销平均仅 0.360 ms，缓存维护单次仅 0.082 ms，二者相对 20 ms 量级的推理都微不足道，差

距并不在 NPU 与内核路径之上。在排除采集与解码争用的固定图口径下，run 阶段的 p50 仅为 Linux 的 1.2 至 1.37 倍，可见 NPU 的计算本身在两侧相近，实时最初配置下 run 升到数倍，是同一小核上多项工作相互争用所致。perf 的计数（较高的 IPC 与很低的缓存缺失率）进一步显示推理受限于计算而非访存。

表 5 Starry 最初配置与 Linux 的实时逐阶段对照（p50，单位 ms）。端到端为自相机帧到达至检测就绪的全链路时延，含解码与帧排队等待，故大于上列计算阶段之和，Starry 在单核堆积下排队等待尤为明显

阶段	Linux	Starry 最初	倍率
letterbox	1.38	75.5	55×
run (NPU)	15.42	71.1	4.6×
outputs_get	1.30	14.3	11×
端到端	25.81	202.9	7.9×

差距集中在单个 A55 小核上的用户态工作，主要是缩放（letterbox）、输出张量重排（outputs_get）与色彩空间转换（decode）这几步。其成因在于线程的放置，第（二）节已说明 Starry 的运行队列在派生与唤醒线程时将其置于当前核且默认没有负载均衡，而机器人程序自身未设置处理器亲和性，于是采集、解码、缩放与推理提交全部落在 cpu0（一个 A55 小核）之上并相互串行。线程放置造成的串行是差距的一层原因，另一层是缩放等步骤尚未用上 RGA 等硬件加速器而仍由 CPU 承担。后续的优化分别针对这两层展开。

7.3 用户态优化

将推理放置到大核

既然默认放置使全部工作串行于一个 A55 小核，将计算量最大的推理迁往 A76 大核是最直接的一步。需要注意放置的时机，NPU 上下文的创建与 USB 的枚举必须在启动核上完成（二者的初始化依赖启动核上的特定状态），否则会挂起，因此亲和性须在模型与相机初始化之后、推理循环之前设置。表 6 给出一次启动内顺次运行的三趟实时对照。

表 6 推理亲和性对端到端时延的影响（640×480 实时，p50，单位 ms）。infer_fps 为推理环节的帧率

阶段	最初	绑定 A55	绑定 A76
run (NPU)	71.1	71.2	24.5
letterbox	75.5	74.9	26.4
outputs_get	14.3	14.3	2.7
端到端	202.9	204.3	65.1
infer_fps	4.3	4.6	8.6

核驻留直方图显示，最初与绑定 A55 时推理全部运行于 cpu0，绑定 A76 时全部运行于 cpu4，可见处理器亲和性在 Starry 上得到遵从。将推理绑定至 A76 后，端到端由 202.9 ms 降至 65.1 ms，约为原先的三分之一。采集帧率亦由 10.9 升至 17.7 fps，原因是推理迁出 cpu0 之后，该 A55 核得以服务相机的传输线程。此时相对 Linux 的 25.81 ms 已收敛到约 2.5 倍，残余差距主要落在 letterbox 的 26.4 ms 之上，它仍是 CPU 缩放。

以整数路径取回输出

机器人所用的单类别检测模型，其输出原本以浮点方式取回。NPU 给出的输出是 INT8 整数，取回时需乘以量化尺度还原为浮点，这一步称为反量化。该模型每帧在输出网格上约有四十万个待还原的数值，其中绝大多数对应没有目标的背景。我们改为以整数方式取回原始输出，先在整数域用阈值筛掉这些背景单元，只对少数存活的单元才做反量化。由于反量化是单调变换，在整数域比较置信度与在浮点域等价，结果不受影响。如此每帧的反量化数量由约四十万降到数十，输出获取由约 12 ms 降至约 1 ms。结合大核放置与 480×640 输入，端到端 p50 达到 11.75 ms（见图 14），且在全部 285 张测试图片上检测结果与浮点路径逐一一致，没有精度损失。这一整数路径的实现细节见第（四）节。

与之配合的还有两处模型层面的准备。一是把模型按 480×640 的实际输入尺寸导出，使其与相机输出一致，相对 640×640 的方形输入省去了填充并减少约四分之一的检测头计算量。

二是把激活函数由 SiLU 改为 ReLU。SiLU 含有 sigmoid 这类非线性运算，在该 NPU 上不被原生高效支持，而 ReLU 是原生的廉价算子，因此 run 阶段约快两倍半，也更易量化；两者的检测精度相当，见表 7。

表 7 网球检测模型的精度（仿真）。mAP@50 为检测精度，召回为查全率，INT8 余弦相似度衡量量化前后输出的一致性，越接近 1 越好

激活	mAP@50	召回	INT8 余弦相似度
SiLU	0.885	0.919	0.998–1.000
ReLU	0.881	0.887	0.998–1.000

以 RGA 承担解码与缩放

在缩放迁往大核之后，把相机帧由 YUYV 转为 RGB 的色彩空间转换成为 CPU 上最高的一项。RGA 既能承担这一转换，也能承担缩放与填充，还能把结果直接写入 NPU 的输入缓冲，从而把原本分离的解码、缩放与输入设置合并为一步。这里走的是真正的零拷贝路径，其前提与实现见第（三）节与第（五）节。经此一步，解码 p50 为 0.776 ms，输入设置降为零，检测结果完全一致，自相机采集到控制指令就绪的整条回路 p50 为 9.65 ms，受相机约 28 fps 的供帧速率限制。

以 JPU 在硬件上解码 MJPEG

参考机器人的相机以 MJPEG 输出，即逐帧 JPEG 压缩的视频流，须先解码为像素才能交由后续处理。前述各步在 YUYV 模式下由 RGA 直接做色彩转换；改用相机原生的 MJPEG 模式后，这一解码若仍由 CPU 承担，单帧约 25 ms，会重新成为时延的主项。接入 JPU 后，MJPEG 由硬件解码为 NV12，单帧约 1 ms，再交 RGA 转色彩并直写 NPU 的输入缓冲，整条 MJPEG→JPU→RGA→NPU 全程零拷贝。由此在机器人实际使用的 MJPEG 模式下，端到端 p50 为 9.06 ms，约 29 fps，吞吐仍受相机供帧速率限制，与 YUYV 路径的 9.65 ms 基本相当。需说明的是，JPU 在此的作用是为相机原生的 MJPEG 模式提供硬件解码，使该步不致退回 CPU 的约 25 ms，而非相对 YUYV 路径的进一步提速，其驱动实现见第（二）节。

把上述优化叠加起来，差距被逐步压下。模型起初以 640×640 的方形尺寸接收输入，在这一尺寸下，Starry 最初配置的端到端 p50 为 202.9 ms，仅将推理放置到 A76 大核就降至 65.1 ms（见表 6），对应的 Linux 基线为 25.8 ms。把模型输入改为与相机输出一致的 480×640 后，既省去了对方形输入的填充，也减少了检测头的计算量。图 14 给出这一输入尺寸下各步优化的端到端时延，最初为 30.6 ms，叠加整数输出路径与大核放置后降至 11.75 ms，与 Linux 在同一输入尺寸下的 11.4 ms 持平；再以 RGA 零拷贝承担色彩转换降至 9.65 ms，最后由 JPU 在硬件上解码相机原生的 MJPEG，端到端 p50 为 9.06 ms，约 29 fps，受相机供帧速率限制。640×640 与 480×640 是同一模型的两种输入尺寸，前者是优化前的方形输入，后者与相机输出一致、省去了填充，各组数应与对应尺寸下的 Linux 基线相比。

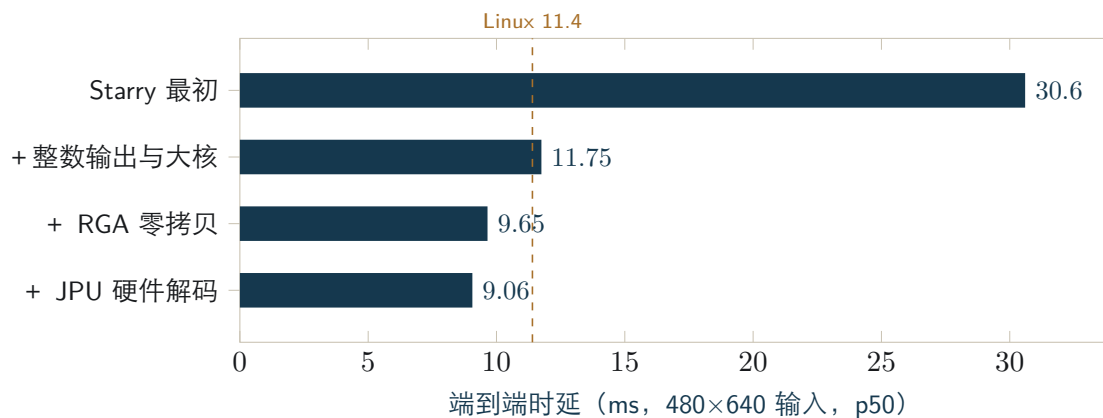


图 14 480×640 输入下各步优化的端到端时延，自最初的 30.6 ms 先降至与 Linux 11.4 ms（赭色虚线）持平的 11.75 ms，再由硬件零拷贝压到约 9 ms（受相机供帧速率限制）。末两条对应 YUYV 与相机原生 MJPEG 两种相机模式，端到端相当，JPU 使 MJPEG 模式得以硬件解码而非相对 YUYV 的提速。横轴只覆盖该输入尺寸，640×640 输入的收敛见表 6

下列三项尝试经测量确认对当前负载没有收益，其原因也指明了各自适用的边界。

- **NPU 张量的零拷贝。**它省去了输入设置的拷贝，却把输出的张量重排从内核侧搬到用户态，在 Starry 的单核上只是把开销换了位置而没有减少，反而把总时延由 217.63 ms 抬高到 266.08 ms。这一手段要在多核并发取走输出的路径下才可能转为收益。
- **第三个 NPU 核。**该模型算子规模较小，逐算子诊断显示其负载几乎集中在一个核上，另两

个核接近空闲，在两核之上已无法继续切分。不过闲置的两个核是一块尚未利用的余量，后续可让三核各处理一帧、以三重缓冲提高吞吐，或在空闲核上并行运行另一个模型以扩展机器人的功能，详见第十二节。

- **锁定 NPU 调频。**它没有带来变化，因为 NPU 在整个运行中已稳定运行在 1.0 GHz。

与 Linux 的整体对照

把优化后的完整流水线放到 Linux 上以同一份程序对照，逐阶段见表 8。在各自的默认调频策略下，Starry 的中位时延更低，但这来自调频策略而非两侧硬件的快慢。Starry 把 NPU、RGA 与 JPU 的时钟直接设为最高且不做动态调频，而 Linux 默认的 `ondemand` 策略在持续负载下会把 NPU 降频。一组固定图的连续推理可看清这一点，Linux 的单次 `run` 由约 4.7 ms 随时间升至约 10.6 ms，而 Starry 始终稳定在约 4.8 ms。也就是说，在同一时钟下两侧的 NPU 推理本就相当，约 4.7 ms，这与 480×640 下 11.75 与 11.4 ms 的持平相互印证；把 Linux 切到最高频策略即可消除中位上的差距。

表 8 完整零拷贝流水线的实时逐阶段对照 (480×640 输入, p50, 单位 ms)。Linux 为其默认的 `ondemand` 按需调频，Starry 不做动态调频

阶段	Linux	Starry
解码 (JPU 与 RGA)	2.11	1.21
run (NPU)	10.89	4.76
outputs_get	0.97	0.73
端到端 (帧到检测)	13.89	9.06

不降频的取舍也有代价，体现在尾部而非中位。Starry 的端到端时延偶有数百毫秒的停顿，最坏一帧达 487 ms，而 Linux 为 22 ms，并因此丢弃约一成的帧；其根源是多核之间的唤醒时延，与第 (二) 节所述的调度方向直接相关，已提交的空闲唤醒竞态修复是其中一步，尾部表明调度仍有工作要做。此外，Starry 的模型冷启动明显更慢，`.rknn` 加载约 2.2 s，而 Linux 约 0.16 s；常驻内存则反而更省，约 19 MB 对 31 MB。在 60 张固定图的校验下两侧检测结果一致，Starry 为 60/60，Linux 为 59/60，平均置信度 0.880 与 0.875。

7.4 内核侧的优化支撑

性能优化中作用于内核侧的部分，关键是让流水线把计算从 CPU 交给专用硬件，并让线程落到合适的核上。前者依赖本队为 Starry 实现的 RGA 与 JPU 两个加速器驱动，二者此前在 Starry 上并不存在，其架构与实现已分别见第 (三) 节与第 (二) 节，它们以 `/dev/rga` 与 `/dev/mpp_service` 对齐既有用户态库的真实接口，使未经改动的机器人程序即可把解码与缩放交给硬件，并经第 (五) 节的 `dma-heap` 共享缓冲构成零拷贝流水线。后者依赖线程放置，本工作以用户态绑核把推理迁往大核解决了当前负载，而更一般的架构感知调度由第 (二) 节的

负载均衡器原型承接。贯穿整个优化过程的测量，则由第（三）节基于硬件 PMU 的 perf 提供。这些内核能力合在一起，构成了上述用户态优化得以成立的基础。

机器人从相机感知到机械臂抓取的完整回路已在 Starry 上跑通，检测结果与 Linux 一致。端到端推理时延在 640×640 输入下由 202.9ms 收敛至 65.1ms，在与相机输出一致的 480×640 输入下达到 11.75ms，与 Linux 处于同一水平。RGA 与 JPU 两个加速器的驱动、基于硬件 PMU 的 perf，以及块设备完成路径的容错等能力都已在内核中实现，可继续支撑 Starry 承载其它边缘智能负载。

8 面向边缘设备的图形与显示支持

一台在野外作业的拾取机器人，最终总要有一块屏幕把它正在看到与正在做的事呈现给人，而要让一套完整的图形界面在 Starry 上跑起来，远不只是画几个像素那么简单。本队成员洪世金选择以 Weston 14 作为目标来检验这件事，它是 Wayland 的参考实现。Wayland 是一套显示服务器协议，用来取代历史悠久但层层叠加的 X 窗口系统，而所谓显示服务器或合成器，是介于内核与各个图形应用之间的那个进程，它独占显示硬件，从各应用收集各自绘制好的窗口缓冲，把它们叠合成一帧整屏画面再交给屏幕，同时把键盘、鼠标与触摸的输入分发回正确的窗口。让这样一个合成器在一套新内核上完整启动并接入一个真实客户端，等于一次性把 Linux 图形与进程间通信栈的一大片都拽上了台面，它对内核的要求横跨显示输出、输入设备发现、套接字传递文件描述符与事件循环四个互不相干的子系统，任何一处不合规，整套界面就立不起来。

合成器依赖的四组内核接口 Weston 的四个子系统各自对应一组内核接口。显示后端 drm-backend 经由 libdrm 去操作 `/dev/dri/card0`，它要协商设备能力、枚举 KMS（内核态显示模式设置）资源、分配显示缓冲，并以原子方式提交一次模式设置与翻页。输入这一面由 libinput 经 libudev 沿 sysfs 遍历去发现 `/dev/input/eventN` 节点，再以一组 evdev ioctl 查询各设备的能力并读取事件。协议服务这一面由 wayland-server 在一个 AF_UNIX 流式套接字上监听，接受客户端连接并接收它们提交的窗口缓冲，其中最吃紧的是要随消息原子地传递文件描述符。最后，整个合成器靠 epoll 与 timerfd 把上述三路事件汇到一个统一的事件循环里轮转。这四面任意一面缺了内核的对应支撑，Weston 要么开不了机，要么开了机也用不了。

缺陷的三个层次与真正的代价 把这次适配遇到的问题摊开看，它们落在三个深浅不同的层次上，而这一分层正是这项工作的主轴。

- **接口缺失。**最浅的一层是接口干脆不存在，打开即以明确的错误码回绝，例如 drm-backend 发现 `/dev/dri/card0` 节点根本没有，evdev 的 `EVIOCGPROP` 与 `EVIOCGABS` 两个 ioctl 未实现，创建 `NETLINK_ROUTE` 套接字直接返回 `EAFNOSUPPORT`，照着错误码补齐即可。
- **貌合神离的隐式约定。**难缠的是第二层，接口看上去都在，编译通过、打开也成功，行为却是错的且不报任何错。`/sys/class/input/` 必须是指向真实设备路径的符号链接，libudev 要靠 `realpath()` 与 `dirname()` 沿它回溯，做成普通目录回溯就在中途断掉；`/dev/input/eventN`

的次设备号必须自 `EVDEV_MINOR_BASE` 即 64 起编, 否则 `libinput` 以 `fstat()` 的 `st_rdev` 反查 `/sys/dev/char/` 时对上不上号; `memfd` 的密封语义、以及套接字传递文件描述符时的字节标记, 都属于这一类约定。这一层的失败是沉默的, `libinput` 只报一句跳过未配置的输入设备, 要查出真因须回头去读用户态库的源码。

- **并发与性能**。最深的一层不挡启动却让人用不下去, `epoll` 的水平触发注册无条件唤醒等待方, 造出每秒约 500 次的空转忙循环, 输入若退回内核节拍驱动会带来约 16 毫秒的延迟, 再叠上单核下中断上下文里的锁争用。

真正昂贵的从来不是写补丁, 而是跨越 `Weston`、`libdrm` 与 `libinput`、`libudev`、`sysfs` 与 `evdev` 直到内核这五层, 把一条条没有写在任何文档里的隐式约定逐一发掘出来。

逐层补齐的内核改动 据此推进的内核改动覆盖了上述每一个面。`DRM/KMS` 这一面补上了 `/dev/dri/card0` 节点, 既支持传统的 `SETCRTC` 路径, 也支持原子化的 `KMS` 模式设置 ([#506](#)), 并以按缓冲粒度分配的 `GEM dumb` 缓冲与带引用计数的 `mmap` 承载像素 ([#514](#))。输入这一面把多个 `eventN` 设备如实暴露出来, 并实现了 `EVIIOCGPROP` 与 `EVIIOCGABS` ([#513](#)), 设备发现所依赖的 `AF_NETLINK` 上的 `rtnetlink`、`uevent` 与 `genl` 几路也一并补全 ([#512](#))。第二层那些隐式约定集中落在一处修订里, `sysfs` 的类目录改为符号链接、`evdev` 次设备号改从 64 起编、并预先在 `/run/udev` 下播下 `udev` 的初始化种子文件, `libudev` 才肯认为设备已配置好 ([#508](#))。`memfd` 的密封不能只守在 `fcntl` 一处, `F_SEAL_SHRINK`、`F_SEAL_GROW` 与 `F_SEAL_WRITE` 的检查被分布到截断、映射与写入各条路径上, 缺一处密封就形同虚设 ([#507](#) 与 [#515](#))。`Weston` 的整体启动收尾、把输入接到中断唤醒以压下延迟、以及让 `AF_UNIX` 流式套接字按字节标记原子地携带控制消息把文件描述符送达, 都在一处合并里完成 ([#509](#))。最深的并发一层, 则由把 `epoll` 的水平触发改为如实排干就绪事件来收敛那条 500 赫兹的空转 ([#504](#)), 并补上 `timerfd` 这一计时事件源 ([#503](#))。

运行与验证 这套支撑最终被验证为可用。`Weston` 以 `DRM` 加 `pixman` 的后端起来后, 一个客户端能连上来, 一个 `GTK4` 程序能把界面绘制出来并经 `VNC` 远程看到, 一个自动化的用例会跑出 `WAYLAND_TEST_PASSED` 这一判据, 并在四种体系结构上一致通过 ([#1160](#))。为防回归, 还引入了以黄金帧做视觉比对的持续集成哨兵守住这条线, 连 `Xwayland` 也跑通了 `xeyes` 这个经典的 `X` 程序, 使既有的 `X` 应用得以在 `Wayland` 之上运行 ([#516](#))。这里必须如实交代它现在所处的位置, 以上全部运行在 `QEMU` 的 `virtio-gpu` 之上, 由 `pixman` 做纯软件合成, 并没有点亮开发板那块物理屏幕上的真实显示控制器, 软件渲染的画面经 `VNC` 间接呈现, 而非由板上的显示硬件直接扫描输出。

面向机器人的延伸方向 把图形栈在 `Starry` 上立起来, 本身是一项与机器人视觉流水线相互独立的系统级贡献, 但它顺带为机器人开出了一条用户侧的方向。有了合成器与窗口客户端这条通路, 就可以在设备上做一个监控与可视化界面, 把实时相机画面连同检测框与机器人状态一起显示出来, 让现场的人直接看到它正在追踪哪一颗球、机械臂处在哪一步。更进一步, `pixman` 的纯 `CPU` 合成完全可以换成用 `RGA` 这块二维加速引擎来做, 而它正是视觉流水线里负责色彩

转换与缩放的同一块硬件（（三）），让显示合成与图像准备复用同一个加速器。这两件事都留作后续工作。



图 15 Weston 14 在 Starry 上以 DRM 加 pixman 软件后端启动, 客户端经 AF_UNIX 套接字连入并提交窗口缓冲, 输入沿 libinput 与 libudev 自 sysfs 与 evdev 发现, 整套事件经 epoll 与 timerfd 汇聚轮转

9 面向 AIOS 的 AI 辅助开发方法

把一个研究型操作系统从能跑做到与 Linux 同档, 涉及大量驱动、系统调用与调优工作。我们在其中系统性地使用了 AI 工具, 并围绕一个问题组织整个流程, 即怎样让 AI 产出真正可靠的操作系统代码, 而不是看似合理却未必正确的代码。我们的做法是把“正确”变成可由机器判定的闸门, 让 AI 的每一处改动都先通过闸门才被接受, 再以真机闭环与实测数据驱动迭代, 而系统架构与最终验收始终在人这一侧。整个过程如图 16 所示。

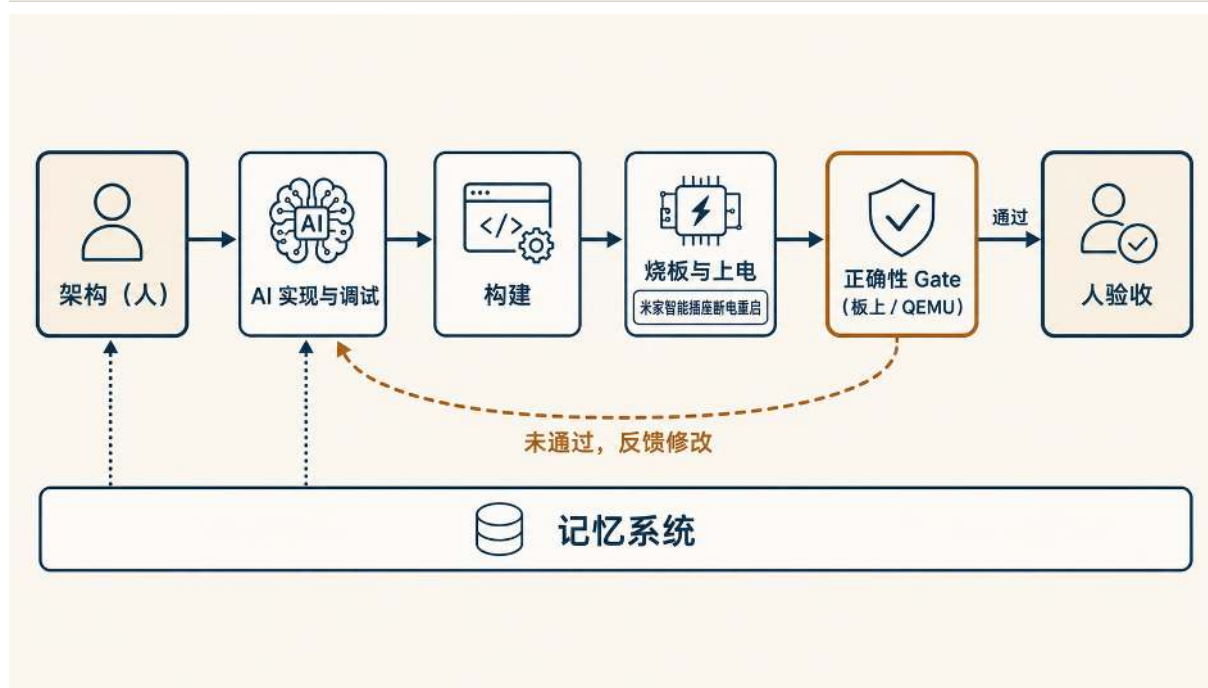


图 16 AI 辅助开发的闭环。人给定规格与架构、并负责最终验收（深色块）；AI 在规格内实现与调试，每次改动须通过板上或 QEMU 的正确性闸门（赭色边框）方可入库，未通过则反馈修改。一只米家智能插座在内核卡死时给开发板断电重启，reboot 系统调用用于切换 Linux 与 Starry，记忆系统在多次会话间承接决策与上下文。

这一方法由几条相互支撑的原则构成。

可机器判定的正确性闸门。操作系统代码的好坏首先在于正确。对驱动与加速器这类工作，我们把正确性落到可逐位核对的基准上，包括 RGA 操作的逐像素 CRC、JPU 解码的校验和、整数输出在 285 张图片上与浮点路径逐一一致、量化模型的余弦相似度，以及 QEMU 与板上的测试用例。AI 的改动只有通过对应闸门才被接受，仅凭看似正确并不能通过。

真机闭环。许多内核问题只在真实硬件上才暴露。我们让迭代直接跑在开发板上，一只米家智能插座负责在内核卡死时给板子断电重启，使一次挂起不至于堵住整条流程；reboot 系统调用（见表 1，上游 PR #1358）则用于在 Linux 与 Starry 之间切换，以同一份可执行文件作对照。由此迭代得以在真机上自动进行。

上述真机闭环在板端通过 starry-board-flow 工具落地，具体流程如图 17 所示。该工具把测试资产和内核镜像的构建与上传、Linux 与 Starry 的切换、串口命令注入、成功标记匹配以及日志回收组织为可复测可组合的自动流程，使真实硬件上的测试验证能够自动高效进行。

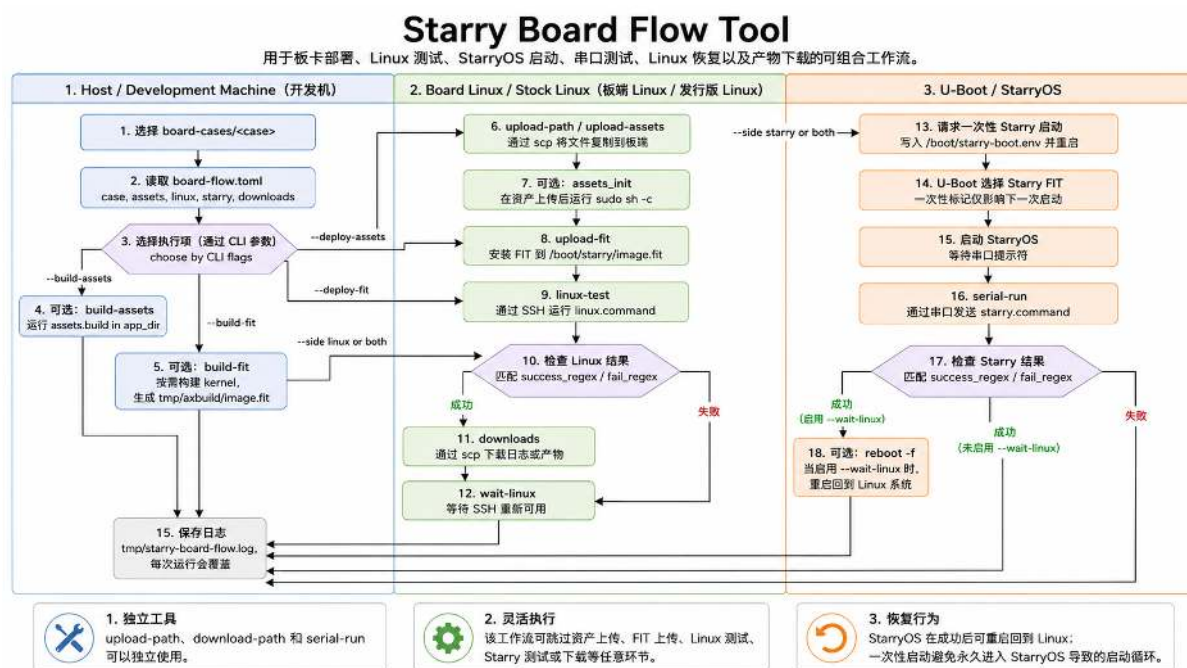


图 17 starry-board-flow 工作流程

测量驱动。优化以第（一）节的剖析口径为准绳，针对实测的逐阶段数据下手而非凭推断。过程中若干基于推断的结论都被实测数据修正，“在同一份可执行文件上超过 Linux”这一设想也在测量面前被否定。

对抗式验证。对每一个性能或正确性主张，我们以独立的复核流程主动尝试反驳，只有经得起反驳的结论才写入结果，从而过滤掉 AI 容易产生的过度断言。

记忆与规格先行。架构与目标由队员以规格的形式给定，一个记忆系统在多次会话之间承接决策与上下文，AI 在规格之内执行，队员审阅。

人掌控架构与验收。上述分工的核心是，AI 加速的是规格之内的实现与调试，而系统如何设计、什么算正确、改动是否采纳，都由队员决定。这也使队员能够在无 AI 辅助的条件下解释关键设计与代码，并在现场完成定位与修改。

按赛事要求，本工作所用的 AI 工具与大模型为 Claude Code（Claude Opus 4.8 与 Sonnet 4.6），以及用于生成示意图的 ChatGPT（GPT-4o 图像生成），使用场景包括资料检索、代码生成、调试定位、日志整理与文档撰写；与 AI 的交互记录予以保留，按工作单元归并的脱敏摘要见提交仓库的 docs/AI 暨狭珍珀瓊萄.md。

10 队员分工

本工作由三名队员分工承担，见表 9。

表 9 队员分工

成员	负责模块与交付物
洪世金	剖析方法与基线、用户态优化、RGA 与 JPU 驱动、硬件 PMU perf、块设备完成路径容错、图形与显示支持（Weston 使能）、报告撰写
王乙凡	USB 串口驱动、reboot 系统调用、PWM 驱动
徐子航	模型训练与数据集准备、系统测试与现场验证

11 开发过程与时间线

本工作的主要开发节点见表 10。

表 10 主要开发节点

阶段	主要工作
图形与显示支持	DRM/KMS、输入、套接字与事件循环，使 Weston 在 Starry 上运行
剖析口径搭建	在同一份程序与内核中补齐观测能力
Linux 基线	在开发板上完成基线的深度剖析
Starry 使能	块设备完成路径容错，首次跑通 NPU 推理
本体接入	USB 串口、reboot 与 PWM 接齐，相机取流，完成拾球
用户态优化	大核放置、整数输出、RGA 零拷贝
内核能力	加速器驱动、硬件 perf 与负载均衡器原型

12 可复现性与后续工作

本工作的环境与复现步骤已整理留存。开发板经直连网络与串口接入主机，基准程序在板上原生构建，Starry 内核在开启相应特性后经 U-Boot 串口引导启动，在无开发板时则经容器中的 QEMU 运行评测，剖析数据以纯标准库渲染为图。完整说明见随附的环境文档。

赛题所关注的性能维度不止单帧时延，还包括启动速度、内存占用与能耗比等。Starry 作为一个以 Rust 实现的精简内核，其常驻内存更省，但模型的冷启动路径明显更慢，整机能耗尚未测量，这几项的实测对照见表 11；缩短冷启动与采集整机能耗均列为后续优化目标。

表 11 多维度指标的 Starry 与 Linux 对照

维度	Linux	Starry	说明
常驻内存（MB）	31	19	无冗余系统服务，约省三分之一
启动至首次推理（s）	0.8	3.6	Starry 的模型加载更慢，待优化
整机功耗	—	—	待板上功耗采集

其余可推进的方向亦较清晰。第（二）节所述的架构感知调度是其中之一，它将依据各核算力自动放置用户态与内核态的线程，免去手动绑核。NPU 的三核可独立工作，而当前的串行点在于设备锁在整个推理期间被持有，一个以每核队列与中断组织的并发完成路径，是进一步提升推理吞吐的结构前提。此外，多核唤醒时延造成的端到端尾部停顿有待随上述调度工作一并收敛；块设备完成中断的底层成因也有待进一步分析，当前的容错处理已能保证稳定挂载。

第八节的图形与显示能力也开出一条面向用户侧的方向。一个在开发板上原生运行的监控界面，可把相机画面、检测框与机器人状态实时呈现给操作者，便于现场调试与演示；而 Weston 的合成当前由 pixman 在 CPU 上完成，可改由视觉流水线已经用到的同一块 RGA 引擎承担，把图层合成也交给硬件。

在机器人应用层面，我们还规划了若干方向。

- 引入 ACT 策略（动作分块的模仿学习策略），让机器人学习并执行更复杂的动作；
- 加入语音输入，使任务可以由语音指令触发；
- 设计更稳健的抓取策略，把抓球的成功率做到接近百分之百；
- 支持里程计，让机器人始终记得回收桶的位置，从而把球准确放入。